COLLEGE OF ENGINEERING TRIVANDRUM (TVE)

Department of Computer Applications

APJ Abdul Kalam Technological University (KTU)

SIGNMATE

AI-ASSISTED INCLUSIVE COMMUNICATION

Master of Computer Applications (MCA) Final-Year Project Presentation
Comprehensive Project Documentation & Comprehensive Viva Preparation Guide

Project Creator:	**Muhammed Rashid KP**
Register / Project ID:	**TVE25MCA-2046**
Degree & Program:	**Master of Computer Applications (MCA)**
Current Implementation:	**Web Application (React 19 + TypeScript + Vite)**
Frontend Deployment:	**signmate-ashy.vercel.app (Vercel Edge)**
Signaling & Data Bus:	**Google Firebase RTDB & Cloud Firestore**
AI Computer Vision Model:	**MobileNetV3Small (27 Classes, 97.00% Test Accuracy)**
Academic Year:	**2025 – 2026**

Confidential Academic Documentation Prepared for Final-Year MCA Viva Voce Examination

Document Generated: October 2026 | Verified Codebase Artifacts Included

Table of Contents

Chapter 1: Project Overview & Core Mission ..... *Mission, Problem Statement, 3 Pitch Formats, Plain-English Definition*

Chapter 2: Complete Technology Stack ..... *Technology Inventory, Status Verification, Computing Taxonomy, Vercel Reality*

Chapter 3: System Architecture & Data Flows ..... *Multi-Tier Architecture, Component Roles, 5 Detailed Data Flows, Control vs Media Plane*

Chapter 4: Project Directory Structure & Core Files ..... *Verified Codebase Tree, File Responsibilities, Step-by-Step Join Execution Flow*

Chapter 5: Requirements, Use Cases & Workflows ..... *Functional/Non-Functional Specs, User Roles, Use-Case & Sequence Diagrams*

Chapter 6: Website Implementation & UI Architecture ..... *React 19 Components, State Hooks, Media Controls, Source Code Inspections*

Chapter 7: WebRTC Video Communication & Mesh Networking ..... *WebRTC Principles, 3-Tier Signaling, 4-Participant Mesh Math, Glare Elimination*

Chapter 8: Machine Learning Dataset & Preprocessing ..... *17,113 Grayscale Images, 27 Classes, Class 0 Background, Double-Normalization Bug*

Chapter 9: Model Training, Transfer Learning & Evaluation ..... *MobileNetV3Small Architecture, 25 ML Concepts, 97% Test Metrics, GRU Transition*

Chapter 10: TensorFlow Lite Deployment & Stabilization ..... *TFLite FlatBuffers (3.88MB), Thresholding, Temporal Smoothing, Debounce Hold Count*

Chapter 11: FastAPI & Backend Inference Architecture ..... *REST Endpoints, Preprocessing Code, CORS Security, Local vs Production Reality*

Chapter 12: Firebase Authentication & Security Boundaries ..... *Auth JWTs, RTDB Ephemeral Paths, onDisconnect Cleanup, Client Key Safety*

Chapter 13: Language Assistance & Dictionary Engine ..... *Client-Side 95k-Word Engine, Prefix Progressions, Gemini Phase 5 Status*

Chapter 14: Hand Tracking & Landmark Visualization ..... *MediaPipe 21 Landmarks, ROI Square Cropping, Anatomical Guard, Canvas Overlay*

Chapter 15: Testing Evidence & Verification Matrix ..... *22 Detailed Test Scenarios, Status Badges, Measured Evaluation Reports*

Chapter 16: Security, Privacy, Ethics & Accessibility ..... *DTLS-SRTP Encryption, Zero Image Retention, Skin-Tone Fairness, WCAG 2.1 AA*

Chapter 17: Production Deployment Configuration ..... *Vercel CI/CD Build Pipeline, Environment Variables, Local Reproduction Guide*

Chapter 18: Project Evolution & Late Stack Migrations ..... *Flutter to Web, ZEGOCLOUD to WebRTC, GRU to MobileNet, Model Defense Answers*

Chapter 19: Known Limitations & Future Roadmap ..... *Honest Technical Boundaries, Continuous Sentence ISL, SFU Scaling Architecture*

Chapter 20: Complete Viva Question Bank (110 Q&As) ..... *110 Categorized Viva Questions (Cats A to T) with Deep Answers & Examiner Warnings*

Chapter 21: Rapid Revision Sheet & Pre-Viva Cheat Sheet ..... *Master Fact Table, 1-Page Emergency Cheat Sheet, 3 Memorizable Presentation Scripts*

Chapter 22: Comprehensive Technical Glossary ..... *63 Essential Technical Terms Defined in Simple, Unambiguous English*

Chapter 1: Project Overview & Core Mission

1.1 Project Identity & Executive Summary

SignMate: AI-Assisted Inclusive Communication is an accessible, web-based real-time video conferencing application designed to facilitate seamless communication between Deaf or Hard-of-Hearing (DHH) signers and hearing individuals. The platform integrates browser-native peer-to-peer WebRTC video calling with an artificial intelligence inference pipeline that detects Indian Sign Language (ISL) static hand gestures, translates them into text, and verbalizes them via speech synthesis.

> **Academic & Institutional Project Identity**
>
> - **Project Title:** SignMate — AI-Assisted Inclusive Communication
> - **Candidate Name:** Muhammed Rashid KP
> - **Student Register Number / Project ID:** TVE25MCA-2046
> - **Degree & Program:** Master of Computer Applications (MCA Final-Year Project)
> - **Institution:** College of Engineering Trivandrum (TVE)
> - **Live Deployed URL:** https://signmate-ashy.vercel.app/

1.2 The Communication Problem

Globally, over 430 million people require rehabilitation for disabling hearing loss, with tens of millions communicating primarily through regional sign languages such as Indian Sign Language (ISL). Despite sign language being a complete, natural language, over 99% of the general hearing population cannot read or comprehend sign gestures.

This fundamental linguistic barrier produces severe everyday challenges:

- **Asymmetry in Everyday Interaction:** Hearing individuals rely heavily on vocal speech and acoustic feedback. When a DHH individual attends a remote medical consultation, academic lecture, or administrative interview, communication stalls without an expensive certified human interpreter.
- **Digital Isolation in Video Conferencing:** Mainstream video calling software (Zoom, Microsoft Teams, Google Meet) is engineered around audio capture and voice-activity detection. When a DHH user signs in a video call, the video platform does not detect voice activity, frequently downgrades video resolution, hides the user from the active speaker layout, and provides zero translation.
- **Typing Fatigue & Disrupted Eye Contact:** DHH individuals often resort to text chat boxes during video meetings. However, constantly looking down to type in a text box breaks interpersonal visual communication, causes cognitive overload, and slows conversational cadence down to an unnatural speed.

1.3 Motivation & Problem Statement

Problem Statement: Existing teleconferencing platforms require Deaf and Hard-of-Hearing users to either secure third-party human interpreters or trade off visual sign communication for cumbersome text chatting. Furthermore, dedicated hardware-based sign interpretation setups (e.g., sensor gloves or depth cameras) are unaffordable, bulky, and inaccessible to everyday users.

Motivation: With modern web standards, any standard consumer smartphone, tablet, or laptop equipped with a commodity RGB webcam can capture video frames and process deep learning models directly in real time. SignMate was conceived to eliminate the interpreter requirement by providing an embedded, zero-cost, browser-accessible inclusive video calling environment.

1.4 Proposed Solution

SignMate delivers a zero-installation, responsive web application that simultaneously hosts up to four participants in an encrypted WebRTC full-mesh video call while executing an intelligent computer vision pipeline. The system crops incoming webcam frames around hand regions, passes them through a deep convolutional neural network (MobileNetV3Small), maps output probabilities across 27 distinct classes, filters out neutral non-gesture frames, temporally stabilizes committed letters, provides intelligent dictionary word suggestions, and synthesizes speech for hearing participants.

1.5 Main Objectives

1. **Zero-Installation Web Accessibility:** Eliminate heavy native application installations by delivering a progressive, responsive web platform running entirely in modern web browsers via Vercel.
2. **Four-Participant WebRTC Video Calling:** Establish encrypted, low-latency, multi-peer audio and video transmission among up to four participants without reliance on commercial proprietary SDKs like ZEGOCLOUD.

3. **High-Accuracy Static Sign Recognition:** Implement transfer-learning based image classification using MobileNetV3Small trained on a 27-class dataset, attaining 97.00% independent test accuracy.
4. **Dual-Mode Real-Time Inference:** Support client-side anatomical fallback using Google MediaPipe Hands alongside a dedicated Python FastAPI inference microservice.
5. **Assistive Communication Features:** Integrate client-side vocabulary recommendations (95,000+ words), screen reading accessibility (ARIA), visual live subtitle streaming, and voice synthesis (Web Speech API).

1.6 Scope & Intended Users

Intended Users: Deaf and Hard-of-Hearing individuals, their hearing family members, remote educators, telemedicine doctors, customer service agents, and peer students collaborating in small study groups.

Scope: The current project scope covers real-time video conferencing for up to 4 participants per room and the classification of *isolated static alphabet hand signs (ISL fingerspelling A–Z)* plus a dedicated background/neutral hand state.

1.7 Critical Clarification: Isolated Alphabet Recognition vs. Continuous Sign Language

> **CRITICAL EXAM CLARIFICATION (DO NOT EXAGGERATE)**
>
> **Examiners will immediately penalize claiming that SignMate translates "continuous Indian Sign Language sentences".**
>
> Indian Sign Language (ISL) is a rich, natural language characterized by dynamic two-handed movements, non-manual facial markers, spatial grammar, and head postures. The implemented SignMate model classifies **isolated static manual alphabet signs (fingerspelling)** from static image frames. It converts fingerspelled gestures into characters (e.g., 'H', 'E', 'L', 'L', 'O'), and assists sentence assembly using client-side word prediction. It does *not* yet interpret full dynamic sentence grammar or facial expressions.

1.8 What Makes SignMate Novel?

Unlike generic video calling software (Google Meet, Zoom, WhatsApp):

FEATURE DIMENSION	STANDARD VIDEO PLATFORMS (ZOOM / MEET)	SIGNMATE PLATFORM
In-Band Sign Recognition	None. Requires external third-party software or human interpreter.	IMPLEMENTED Embedded computer vision pipeline processes webcam frames live.
Speech Output for Signs	None. DHH users must manually type into the text box.	IMPLEMENTED Recognized text is automatically synthesized to speech for hearing callers.
Dynamic Subtitle Broadcasting	Only transcribes spoken English voice. Ignores signed gestures.	IMPLEMENTED Signs are broadcasted live over the WebRTC data layer as visual captions.
Dictionary Prefix Expansion	Standard text auto-correct only in typing chat.	IMPLEMENTED Single-click expansion of fingerspelled letters into full conversational words.
Infrastructure Cost	Expensive enterprise licenses for breakout interpretation rooms.	IMPLEMENTED 100% open-source, serverless frontend, peer-to-peer mesh.

1.9 Three Pitch Formats for Presentation Day

> **1. The 30-Second Examiner Pitch**
>
> "Respected examiners, **SignMate** is an inclusive web-based video conferencing platform built for Deaf and Hard-of-Hearing individuals. It allows up to four participants to communicate over WebRTC while an integrated lightweight deep learning model, MobileNetV3Small, detects Indian Sign Language alphabet gestures directly from webcam frames with 97% test accuracy. The system translates manual signs into live subtitles, offers conversational word suggestions, and synthesizes speech in real time, bridging the barrier between signers and hearing callers without requiring human interpreters."

2. The 1-Minute Comprehensive Pitch

"Millions of Deaf and Hard-of-Hearing individuals face severe isolation in modern remote communication because conventional tools like Zoom or Google Meet are designed solely around spoken audio. SignMate solves this by embedding computer vision directly into a peer-to-peer WebRTC video room.

Built using React 19, TypeScript, and Tailwind CSS on the frontend, SignMate captures live camera frames and processes them through MobileNetV3Small, a convolutional neural network trained on 17,113 grayscale images across 27 classes—including a crucial neutral background class. The platform incorporates temporal smoothing and debouncing to prevent accidental character repetitions, provides instant dictionary word suggestions from a 95,000-word dataset, and speaks the words out loud using the Web Speech API. It is already deployed live on Vercel and accessible across any modern web browser without installing external plugins."

3. The 2-Minute Technical Introduction

"From an architectural perspective, SignMate is engineered as a decoupled, multi-tier system. The presentation tier is a responsive single-page web application developed with React 19, Vite, and TypeScript. Video communication is powered by a WebRTC full-mesh topology accommodating up to four active participants, coordinated through a resilient three-tier signaling architecture utilizing Firebase Realtime Database with BroadcastChannel fallback for multi-tab testing.

For machine learning inference, the application utilizes a hybrid execution model. At its core is `SignMate_MobileNetV3Small`, a transfer-learning model trained on ImageNet weights with a custom classification head (Dense 128 + Dropout + Softmax 27). The model receives 160×160 single-channel grayscale inputs, expands them via internal concatenation, and outputs class confidence scores. In local development or client environments, this model is served via a high-performance Python FastAPI microservice at sub-50ms latencies or deployed as an optimized 3.88 MB TensorFlow Lite model. Client-side, Google MediaPipe Hands provides hand region-of-interest cropping and anatomical landmark verification. The system is secured using Firebase Authentication, HTTPS transit encryption, and strict input validation."

4. "Explain Your Project In Your Own Words" (Natural, Confident Viva Answer)

"I built SignMate because if a Deaf person and a hearing person want to have a video call today, they either need to hire an interpreter or constantly look down and type in a chat box. SignMate brings the interpreter inside the call. When a user shows a sign to the camera, the website recognizes the letter, suggests full words like 'HELLO' or 'THANK YOU', displays it as live captions for both people, and even speaks the words out loud for the hearing person. I deployed it as a website so anyone can open a link on their phone or laptop and start talking immediately."

Chapter 2: Complete Technology Stack

2.1 Comprehensive Technology Inventory & Status Table

The table below documents every core technology, tool, library, and service evaluated and utilized within the SignMate project. Every component is classified according to its verified operational status in the repository.

TECHNOLOGY	PURPOSE & ROLE	WHERE IT IS USED	WHY SELECTED	COMMUNICATION LINK	CURRENT STATUS
React 19	Frontend UI Library	Entire Web Client (`src/`)	Modern declarative component-based UI, efficient Virtual DOM reconciliation, seamless state hooks.	Renders DOM, binds event listeners, interacts with WebRTC and AI services via custom hooks.	**IMPLEMENTED & VERIFIED** Active in production build.
TypeScript 5/6	Static Typing & Code Safety	All source scripts (`.ts, .tsx`)	Catches type mismatches at compile time, ensures reliable data contracts across WebRTC and AI layers.	Transpiled by Vite/ESBuild into clean standard JavaScript for browser runtime execution.	**IMPLEMENTED & VERIFIED** Strict typing enforced.
Vite 8	Frontend Build Tool & Dev Server	Root build pipeline (`vite.config.ts`)	Lightning-fast Hot Module Replacement (HMR) and optimized Rollup-based production bundling.	Serves client assets on port 5173 locally; compiles static artifacts into `dist/` for Vercel.	**IMPLEMENTED & VERIFIED** Production builds functional.
Tailwind CSS 3	Utility-First Styling	Entire UI layout & components	Rapid responsive styling, consistent dark-mode design system, zero runtime style overhead.	Compiled into atomic CSS classes at build time by PostCSS.	**IMPLEMENTED & VERIFIED** Active across all pages.
WebRTC (Native Browser API)	P2P Audio/Video/Data Communication	`src/services/WebRTCService.ts` , `RoomPage.tsx`	Zero-plugin browser standard, sub-second latency, hardware-accelerated VP8/H.264 video pipelines.	Direct peer-to-peer UDP media exchange between participant browsers via ICE/DTLS/SRTP.	**IMPLEMENTED & VERIFIED** 4-peer full mesh active.
Firebase Realtime Database (RTDB)	WebRTC Signaling & Room State Sync	`WebRTCService.ts` , `config.ts`	Millisecond-latency WebSocket synchronization for SDP offers, answers, and ICE candidate exchange.	Browser connects to Google cloud servers via secure WSS to read/write ephemeral signaling paths.	**IMPLEMENTED & VERIFIED** Multi-peer signaling active.
Firebase Authentication	User Identity & Session Management	`src/firebase/firebaseAuth.ts` , `LoginPage.tsx`	Secure credential storage, token management, OAuth support, zero backend auth maintenance.	Frontend talks to Google Identity Toolkit REST API; maintains client auth state tokens.	**IMPLEMENTED & VERIFIED** Email/password & Google login.
Cloud Firestore	NoSQL Document Database	`src/firebase/firestoreRooms.ts`	Persistent storage for user profiles and registered meeting rooms.	Client SDK communicates directly with Google Firestore endpoints using secure rules.	**IMPLEMENTED & VERIFIED** Active for room persistence.

TECHNOLOGY	PURPOSE & ROLE	WHERE IT IS USED	WHY SELECTED	COMMUNICATION LINK	CURRENT STATUS
MobileNetV3Small (Keras / TensorFlow)	Deep Learning Sign Classifier	`SignMate_stage1_best.keras`, `fastapi_server.py`	State-of-the-art mobile computer vision architecture; high efficiency (~1.17M params), high accuracy.	Accepts [1, 160, 160, 1] input from image crop; outputs 27-element probability vector.	**IMPLEMENTED & VERIFIED** 97% test accuracy checkpoint.
TensorFlow Lite (TFLite)	Edge Runtime ML Model	`SignMate_MobileNetV3Small.tflite`	Compact (3.88 MB) flatbuffer model optimized for low-latency, low-memory embedded inference.	Loaded directly via TensorFlow Lite runtime on CPU/GPU delegates.	**IMPLEMENTED & VERIFIED** Exported & validated (3.88 MB).
FastAPI & Uvicorn (Python 3.12/3.13)	High-Performance ML Inference Microservice	`fastapi_server.py`	Asynchronous, lightweight Python web framework; handles numpy tensors and model execution.	Exposes REST endpoints (`/api/v1/predict`, `/api/v1/health`) accessed via HTTP POST.	**IMPLEMENTED & VERIFIED** Serves local predictions.
Google MediaPipe Hands	Hand Localization & 21 3D Landmarks	`src/ai/handSignDetector.ts`, `HandLandmarkOverlay.tsx`	Real-time hand tracking; computes square bounding boxes and validates anatomical finger states.	Runs client-side in browser; provides ROI crop fed into neural network.	**IMPLEMENTED & VERIFIED** Client-side tracking active.
Dictionary Recommendation Engine	Contextual Vocabulary Assistance	`src/services/dictionaryService.ts`, `conversational_words.json`	High-speed prefix tree lookup across 95,000+ words; prioritizes common conversational terms.	In-memory JavaScript search executed synchronously on client thread (<2ms).	**IMPLEMENTED & VERIFIED** 95k word dictionary active.
Google Gemini API	Generative Language Completion Proxy	`src/gemini/geminiSuggestionsService.ts`	Designed for sentence-level conversational auto-completion from recognized sign sequences.	Intended to be proxied through secure backend to shield API keys from browser client.	**PARTIALLY IMPLEMENTED (PHASE 5)** Interface defined; pending proxy.
Vercel Edge Network	Global CDN Hosting Platform	Production Web Deployment	Zero-configuration Git CI/CD, automatic HTTPS, global edge distribution, ultra-fast TTFB.	Delivers static HTML, CSS, JavaScript, and asset bundles to web visitors.	**IMPLEMENTED & VERIFIED** Live at signmate-ashy.vercel.app.

2.2 Architectural Taxonomy: Distinguishing Core Computing Concepts

During project viva examinations, candidates are frequently asked to define and contrast computing terminology. The following definitions clarify how these layers function in SignMate:

- **Framework (e.g., FastAPI):** A foundational software environment providing an architectural skeleton. It controls the application flow (Inversion of Control—*"the framework calls your code"*). FastAPI dictates how routing, dependency injection, and middleware operate.
- **Library (e.g., React, Lucide Icons):** A curated collection of pre-written functions and classes that you call directly from your code (*"your code calls the library"*). React provides UI component primitives without dictating your entire backend architecture.
- **API (Application Programming Interface, e.g., `/api/v1/predict`, Web Speech API):** A formalized contract of endpoints, methods, and schemas enabling two distinct software components to communicate.
- **Protocol (e.g., WebRTC, HTTP/HTTPS, WebSocket):** A standardized set of network rules governing data serialization, formatting, error-checking, and transmission across communication links.
- **Database (e.g., Firebase RTDB, Cloud Firestore):** An organized, queryable persistence engine capable of storing, indexing, and retrieving structured application records.
- **Model (e.g., MobileNetV3Small):** A trained mathematical artifact consisting of neural layer topologies and numerical weight matrices that maps input features to predicted output probabilities.
- **Hosting Platform (e.g., Vercel):** Cloud server infrastructure that stores compiled website artifacts, executes domain routing, issues SSL certificates, and serves files over high-speed CDNs.

2.3 Why a Web Application Over a Flutter Mobile App?

An earlier iteration of this project investigated Flutter. However, the final production implementation was deliberately migrated to a modern React 19 web application for vital academic and practical reasons:

1. **Zero-Barrier Instant Accessibility:** A Deaf user or hearing caller does not need to download an APK, install third-party stores, or grant OS-level installation privileges. They simply click a link (`signmate-ashy.vercel.app/room/X8LN7A`) and join instantly.
2. **Cross-Platform Universal Compatibility:** A web application runs identically across Windows, macOS, Linux, Android, iOS, ChromeOS, and Smart Displays without compiling separate native toolchains (Xcode, Android Gradle).
3. **Immediate Continuous Deployment:** Bug fixes, UI updates, and dictionary enhancements pushed to Git are live on Vercel in 45 seconds, requiring zero app-store review delays or manual user updates.
4. **Web Standards Integration:** Modern browsers natively support WebRTC, WebGL, Web Audio, and Web Speech APIs without wrapping them in fragile mobile plugin bridges.

2.4 What Vercel Hosts vs. Where the Python Backend Runs

CRITICAL EXAM UNDERSTANDING: DEPLOYMENT SEPARATION

Vercel hosts the static frontend build artifacts: the compiled HTML, CSS, JavaScript chunks, images, and static dictionaries (`conversational_words.json`). Vercel is a serverless static/Node CDN platform. It *does not* run long-running Python processes or host heavy GPU TensorFlow inference servers.

Where does the Python FastAPI backend run? The FastAPI backend (`fastapi_server.py`) runs on a host machine (e.g., local workstation, Docker container, or virtual private server such as Render, AWS EC2, or Railway) listening on port `8000`. When users access the live Vercel website without a live connected Python server, SignMate seamlessly falls back to client-side Google MediaPipe landmark processing and client-side dictionary assistance, ensuring the website never crashes or displays a blank screen.

Chapter 3: System Architecture & Data Flows

3.1 High-Level End-to-End System Architecture

SignMate employs a distributed, decoupled multi-tier architecture designed for low latency, fault tolerance, and universal web accessibility. The system separates the user presentation, real-time media transport, signaling coordination, and machine learning inference into independent subsystems.

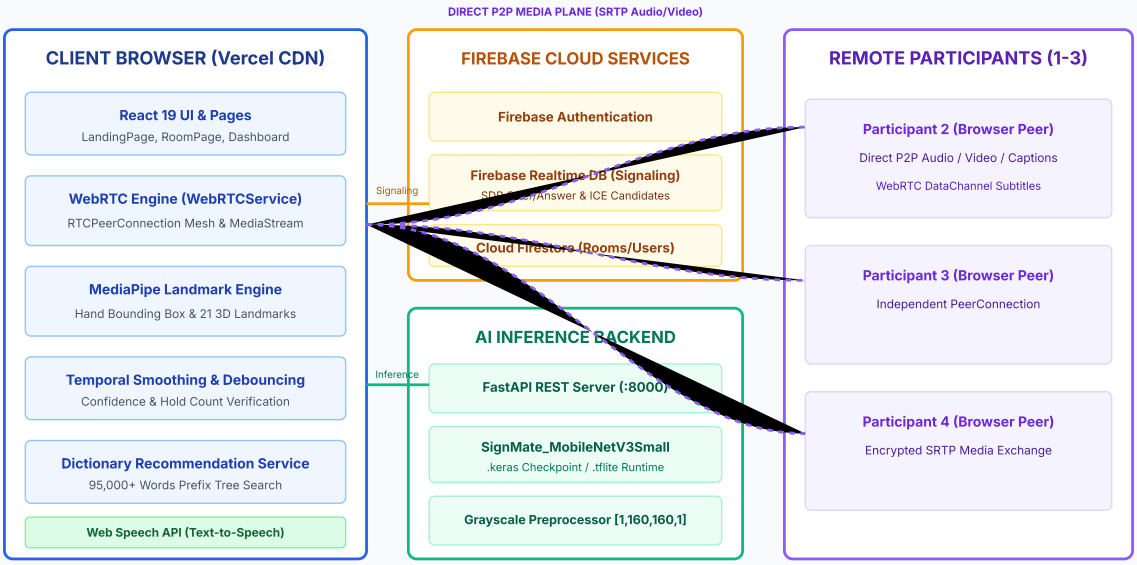


Figure 3.1: SignMate End-to-End System Architecture Diagram

3.2 Component Responsibility Table

COMPONENT	PRIMARY RESPONSIBILITY	INPUTS	OUTPUTS
React Frontend (`src/`)	Renders responsive UI, captures webcam/microphone streams, manages call layout, triggers speech synthesis.	User interactions, camera hardware frames, remote media streams.	Rendered DOM, canvas crops, local media tracks, speech synthesis utterances.
WebRTC Service (`WebRTCService.ts`)	Coordinates peer connections, handles SDP offer/answer exchange, manages ICE candidates, broadcasts live sign subtitles.	Room ID, local MediaStream, signaling messages from Firebase RTDB.	Encrypted P2P video/audio tracks, remote streams, real-time message events.
Hand Sign Detector (`handSignDetector.ts`)	Tracks hand regions using MediaPipe, calculates 1:1 square bounding box, dispatches inference to FastAPI or executes geometric fallback.	HTMLVideoElement raw camera frames.	Bounding box, 21 landmarks, detected letter, confidence score, inference latency.
FastAPI Microservice (`fastapi_server.py`)	Loads `SignMate_stage1_best.keras`, normalizes incoming image frames to [1, 160, 160, 1] float32 grayscale, executes prediction.	Base64 JPEG or multipart image frames.	Predicted class index (0–26), letter label ('A'–'Z' or ''), confidence, top-5 probabilities.
Dictionary Service (`dictionaryService.ts`)	Maintains in-memory vocabulary (95,000+ words), provides instant prefix-filtered word suggestions, prioritizes conversational phrases.	Recognized character buffer prefix (e.g., 'HEL').	Array of recommended words (e.g., ['HELLO', 'HELP', 'HEALTH']).
Firebase Authentication	Authenticates users, encrypts passwords, issues JWT access tokens, persists user sessions across browser reloads.	User email, password, or Google OAuth credentials.	Authenticated User object, UID, display name, session tokens.
Firebase Realtime Database	Serves as the high-speed signaling bus for WebRTC connection establishment and room presence tracking.	SDP offers, SDP answers, ICE candidates, disconnect hooks.	Real-time synchronization events pushed to connected peers.

3.3 Five Core Data-Flow Scenarios

A. User Registration and Login Flow

1. The user navigates to `/register` or `/login` and submits their credentials (email and password) or selects Google Sign-In.
2. `firebaseAuth.ts` invokes `signInWithEmailAndPassword()` or `signInWithPopup()` directly via HTTPS to Google Identity Toolkit.
3. Firebase verifies credentials, generates a cryptographically signed JWT token, and caches it in browser `indexedDB`.
4. The React `useAuth()` hook updates the application state, dismissing login loaders and redirecting the user to `/dashboard`.

B. Creating and Joining a Video Room

1. On the dashboard, a user clicks "Create Room". `firestoreRooms.ts` generates a 6-character room code (e.g., `X8LN7A`) and writes a document to Firestore under `rooms/X8LN7A`.
2. The creator is routed to `/room/X8LN7A`. The browser activates `navigator.mediaDevices.getUserMedia()` to acquire the local camera and microphone.
3. A second user enters `X8LN7A` into the "Join Room" modal. The app queries Firestore to verify room existence and check that participant capacity has not exceeded the 4-participant limit.
4. Both users now share the same room identifier in Firebase Realtime Database.

C. Four-Participant Video Call (Full Mesh)

1. As each participant joins, their client registers their unique peer ID in RTDB at `rooms/X8LN7A/peers/{clientId}`.
2. When 4 users are present (Users A, B, C, D), deterministic ordering (`clientId_A < clientId_B`) determines who initiates the offer.
3. Each participant establishes 3 separate `RTCPeerConnection` instances, resulting in exactly $N(N-1)/2 = 6$ direct peer-to-peer media links.
4. Audio and video streams flow directly between browsers over UDP; no media data passes through Vercel or Firebase servers.

D. AI Sign Recognition Pipeline

1. In an active call, the user turns on the "Sign AI" toggle button. The `useSignAI` hook triggers a 120ms frame evaluation loop.
2. Google MediaPipe Hands scans the video frame. If a hand is visible, it extracts 21 3D landmarks and calculates a 1:1 square bounding box.
3. An offscreen 160×160 canvas crops the hand, converts it to grayscale, and dispatches a lightweight JSON payload to `fastapi_server.py`.
4. The neural network computes a forward pass, outputting 27 class scores. If confidence exceeds 0.70 and is stable across 2 consecutive frames, the letter is committed.
5. The recognized letter is broadcasted to all participants via WebRTC data channels and converted to speech via `window.speechSynthesis`.

E. Language / Dictionary Suggestions Flow

1. As letters are committed (e.g., 'H', then 'E', then 'L'), `dictionaryService.ts` reads the active prefix.
2. In less than 2 milliseconds, the client-side prefix matcher queries `dictionary_words.json` and prioritizes high-frequency conversational terms from `conversational_words.json`.
3. The UI presents clickable suggestion chips (e.g., [HELLO], [HELP], [HEALTH]).
4. Clicking a chip replaces the partial fingerspelled sequence with the complete word and appends a space, ready for the next word.

3.4 Control Plane vs. Media Plane (Simple Examiner Explanation)

> **VIVA ESSENTIAL: CONTROL PLANE VS. MEDIA PLANE**
>
> **The Control Plane (Signaling):** Responsible for setting up, coordinating, and terminating the call. It handles *lightweight metadata*: who wants to talk to whom, what codecs are supported (SDP offers/answers), and what public IP addresses are available (ICE candidates). In SignMate, the control plane runs over Firebase Realtime Database and WebSockets.
>
> **The Media Plane (Audio/Video Transport):** Responsible for transmitting the actual, heavy video and audio packets. Once the control plane finishes exchanging IP addresses and cryptographic keys, media flows *directly between user browsers* using the Secure Real-Time Transport Protocol (SRTP) over UDP. No media data touches the web server.

Chapter 4: Project Folder Structure & Source Code Inspection

4.1 Verified Project Directory Tree

The tree below reflects the exact physical directory layout of the SignMate project inspected directly from the root workspace:

```
SignMate/
├── dist/                              # Compiled production build assets for Vercel deployment
├── public/                            # Static public web assets
│   ├── conversational_words.json      # 104 KB: Curated high-frequency English conversational words
│   ├── dictionary_words.json          # 1.19 MB: Comprehensive English dictionary dataset
│   ├── favicon.svg                    # SVG brand favicon for browser tabs
│   ├── icons.svg                      # Scalable SVG UI icon sprite sheet
│   └── signmate_labels.json           # 27 class mappings (0 = background/neutral, 1-26 = A-Z)
├── src/                               # React 19 + TypeScript Application Source Code
│   ├── ai/                            # Machine Learning and Computer Vision subsystems
│   │   ├── fastApiAIService.ts        # Client bridge for calling Python FastAPI backend
│   │   ├── fastApiConfig.ts           # Endpoint URLs, class labels, and localhost/LAN resolvers
│   │   └── handSignDetector.ts        # Dual-mode MediaPipe + MobileNetV3Small frame processor
│   ├── assets/                        # Static UI graphic assets (hero.png, logos)
│   ├── auth/                          # Authentication components (SocialAuthButtons.tsx)
│   ├── components/                    # Reusable UI component library
│   │   ├── ai/                        # AI overlay, landmark canvas, recognition panels
│   │   │   ├── AIControl.tsx          # Toggle switches for sign recognition
│   │   │   ├── HandLandmarkOverlay.tsx# Canvas overlay rendering 21 hand joints & skeletons
│   │   │   └── RecognitionPanel.tsx   # Visual output card showing letters & word chips
│   │   ├── brand/                     # Brand identity (SignMateLogo.tsx)
│   │   ├── user/                      # User dashboard navigation (UserSidebar.tsx)
│   │   ├── video/                     # Video call layout (VideoGrid, ParticipantTile, RoomHeader)
│   │   ├── Button.tsx                 # Accessible button component with variant states
│   │   ├── ErrorBoundary.tsx          # React error boundary catching runtime UI exceptions
│   │   ├── Modal.tsx                  # Accessible dialog modal component
│   │   └── Navbar.tsx                 # Global top navigation bar with auth status
│   ├── firebase/                      # Firebase Cloud Services SDK integration
│   │   ├── config.ts                  # App initialization for Auth, RTDB, and Firestore
│   │   ├── firebaseAuth.ts            # Email/Password & Google Sign-In authentication adapter
│   │   └── firestoreRooms.ts          # Firestore room creation, joining, and persistence adapter
│   ├── gemini/                        # Gemini Generative AI Language Assistance
│   │   ├── geminiConfig.ts            # Gemini proxy configuration schema
│   │   └── geminiSuggestionsService.ts# Planned Phase 5 language suggestion adapter
│   ├── hooks/                         # Custom React Hooks
│   │   ├── useAuth.tsx                # Context hook managing active user session
│   │   ├── useMediaDevices.ts         # Camera/microphone hardware permissions & track state
│   │   ├── useSignAI.ts               # Sign recognition state, debouncing, & speech synthesis
│   │   ├── useSocial.tsx              # Friends & social directory hook
│   │   └── useToast.tsx               # Non-blocking notification banner hook
│   ├── pages/                         # Application routes and views
│   │   ├── common/SecurityPage.tsx    # Platform privacy, encryption, & security audit page
│   │   ├── user/                      # User management views
│   │   │   ├── UserOfflineDetectionPage.tsx # Standalone offline sign practice & speech practice
│   │   │   ├── UserProfilePage.tsx    # User account settings and preferences
│   │   │   └── UserSupportPage.tsx    # Frequently asked questions & feedback mailer
│   │   ├── DashboardPage.tsx          # Post-login hub: create room, join room, offline mode
│   │   ├── LandingPage.tsx            # Marketing homepage, accessibility overview, feature showcase
│   │   ├── LoginPage.tsx              # User login screen with validation
│   │   ├── RegisterPage.tsx           # Account registration screen
│   │   └── RoomPage.tsx               # 74 KB: Core WebRTC 4-participant video conferencing room
│   ├── rooms/                         # Room modals (CreateRoomModal, JoinRoomModal)
│   ├── services/                      # Decoupled service layer
│   │   ├── AIService.ts               # Interface definition for AI recognition services
│   │   ├── AuthService.ts             # Interface definition for user authentication
│   │   ├── dictionaryService.ts       # 95,000-word prefix recommendation engine
│   │   ├── GeminiService.ts           # Abstract interface for generative completions
│   │   ├── RoomService.ts             # Interface definition for room lifecycle
│   │   └── WebRTCService.ts           # 26 KB: 3-tier signaling & 4-peer full mesh WebRTC engine
│   ├── types/                         # TypeScript interfaces (ai.ts, auth.ts, room.ts, video.ts)
│   ├── utils/                         # Helper utilities (a11y.ts, roomCode.ts, validators.ts)
│   ├── App.tsx                        # Main application router and root provider wrapper
│   ├── index.css                      # Global Tailwind CSS definitions & color tokens
│   └── main.tsx                       # React DOM root entrypoint
├── .env                               # Local environment variables (Firebase keys, API URLs)
├── database.rules.json                # Firebase Realtime Database security rules
├── dictionary_compact.json            # 22.4 MB: Source dictionary word dataset (95k+ terms)
├── fastapi_server.py                  # 11.8 KB: Production Python FastAPI AI inference microservice
├── firebase.json                      # Firebase project configuration manifest
├── firestore.rules                    # Cloud Firestore document security rules
├── package.json                       # Node dependencies (React 19, Vite 8, Firebase 12, Lucide)
├── signmate_labels.json               # Model class label dictionary (classes 0 to 26)
├── SignMate_MobileNetV3Small.tflite   # 3.88 MB: Converted TensorFlow Lite edge deployment model
├── SignMate_stage1_best.keras         # 5.27 MB: Trained MobileNetV3Small deep learning checkpoint
└── vite.config.ts                     # Vite build tool and plugin configuration
```

4.2 Critical File Breakdown & Responsibilities

1. `fastapi_server.py` (11.8 KB)

Role: High-performance Python backend server powering real-time deep learning inference.
Why it exists: To load the heavy TensorFlow/Keras neural network (`SignMate_stage1_best.keras`) and execute hardware-accelerated tensor computations.
Key Functions:

- `load_model_and_labels()`: Deserializes the Keras model using compatibility patches for legacy Keras 3 layers (`RandomContrast`, `Dense`) and parses `signmate_labels.json`.
- `preprocess_image(image_bytes)`: Converts arbitrary image inputs into single-channel grayscale `[1, 160, 160, 1]` float32 arrays scaled between 0 and 255.
- `POST /api/v1/predict/base64`: Receives base64 webcam frames, executes `model.predict()`, filters out background class 0, and returns predicted letters with top-5 confidence distributions.

2. `src/services/WebRTCService.ts` (26.3 KB)

Role: Core WebRTC video, audio, and subtitle communication engine.
Why it exists: Manages browser peer connections without paying for commercial video conferencing SDKs.
Key Functions:

- `getOrCreatePeer(remoteId)`: Lazily instantiates and configures a dedicated `RTCPeerConnection` with Google STUN servers for every remote peer in the room.
- `setupRealtimeDatabaseSignaling(roomId)`: Connects to Firebase Realtime Database at `rooms/{roomId}/mesh/{pairKey}` to exchange SDP offers, answers, and ICE candidates.
- `sendLiveCaption(letter, text)`: Broadcasts real-time fingerspelled characters and accumulated sentences across peers via both WebSockets and BroadcastChannel.

3. `src/ai/handSignDetector.ts` (32.8 KB)

Role: Client-side computer vision orchestrator and fallback engine.
Why it exists: Connects video hardware to AI models while guaranteeing uninterrupted local operation if the Python server is offline.
Key Functions:

- `initMediaPipe()`: Dynamically loads Google MediaPipe Hands from CDN to track hand coordinates.
- `detectSign(video)`: Computes 1:1 square bounding boxes around the hand, crops the frame using an offscreen canvas, checks FastAPI health, and dispatches inference requests.
- `classifyLandmarks(landmarks)`: 3D geometric rule engine that verifies finger angles and provides instant local fallback predictions if the neural network is unreachable.

4. `src/services/dictionaryService.ts` (6.1 KB)

Role: Ultra-fast in-memory vocabulary prediction engine.
Why it exists: Enables signers to fingerspell only the first 2–3 letters of a word (e.g., 'TH') and immediately select the full word ('THANK YOU') instead of laboriously signing every letter.
Key Functions:

- `init()`: Asynchronously caches `conversational_words.json` into browser memory.
- `getWordSuggestions(prefix, maxResults)`: Performs case-insensitive prefix tree matching, returning the top 8 conversational words in under 2 milliseconds.

5. `src/pages/RoomPage.tsx` (74.8 KB)

Role: The primary video conferencing view where users meet and communicate.
Why it exists: Assembles the video grid, participant tiles, call controls (mute, camera, screen share), live subtitle drawers, AI landmark overlays, and voice synthesis into a cohesive responsive layout.

4.3 End-to-End Execution Flow: Opening the Website & Joining a Call

1. **Initialization (`main.tsx` → `App.tsx`):** The browser loads HTML/JS bundle from Vercel. `FirebaseApp` initializes Auth, Firestore, and RTDB singletons. React Router renders `LandingPage.tsx`.
2. **Authentication Check (`useAuth.tsx`):** The app verifies if an active session exists in `indexedDB`. If authenticated, the user accesses `DashboardPage.tsx`.
3. **Room Navigation:** The user enters room code `X8LN7A`. React Router navigates to `/room/X8LN7A`, mounting `RoomPage.tsx`.

4. **Hardware Acquisition (`useMediaDevices.ts`):** The browser prompts for camera and microphone permissions via `navigator.mediaDevices.getUserMedia({ video: true, audio: true })`. The resulting `MediaStream` is bound to `localVideoRef`.
5. **Signaling & Mesh Establishment (`WebRTCService.ts`):**
   - The local client registers its unique ID (`peer_abc123`) in Firebase RTDB at `rooms/X8LN7A/peers/peer_abc123`.
   - If a peer (`peer_xyz789`) is already in the room, deterministic comparison initiates an SDP offer.
   - Both browsers exchange ICE candidates via RTDB and establish direct UDP media flow.
6. **Sign Recognition Activation (`useSignAI.ts`):** The user enables "Sign AI". `handSignDetector.ts` begins analyzing video frames every 120ms, feeding hand crops to `fastapi_server.py` or MediaPipe, committing recognized letters, updating live subtitles, and speaking words through the speaker.

Chapter 5: Requirements, Use Cases & Workflow Modeling

5.1 Functional Requirements (FR)

REQ ID	REQUIREMENT DESCRIPTION	TARGET COMPONENT	VERIFICATION STATUS	ACCEPTANCE CRITERIA
FR-01	User Account Registration & Login	`firebaseAuth.ts`, `LoginPage.tsx`	IMPLEMENTED & VERIFIED	Users can register with email/password or Google OAuth; sessions persist across reloads.
FR-02	Room Creation with Unique Code	`firestoreRooms.ts`, `DashboardPage.tsx`	IMPLEMENTED & VERIFIED	Generates a random 6-character alphanumeric room code and persists room record.
FR-03	Room Joining & Capacity Enforcement	`RoomPage.tsx`, `WebRTCService.ts`	IMPLEMENTED & VERIFIED	Validates room code; rejects entry if the room has reached 4 participants.
FR-04	Browser Camera & Mic Access	`useMediaDevices.ts`	IMPLEMENTED & VERIFIED	Requests standard browser permissions; cleanly handles device denial or disconnections.
FR-05	Multi-Participant WebRTC Video Mesh	`WebRTCService.ts`, `RoomPage.tsx`	IMPLEMENTED & VERIFIED	Establishes P2P audio and video streams between up to 4 participants simultaneously.
FR-06	Media Controls (Mute / Video Toggle)	`CallControls.tsx`, `RoomPage.tsx`	IMPLEMENTED & VERIFIED	Participants can toggle audio and video tracks without severing the peer connection.
FR-07	Real-Time Sign Gesture Recognition	`handSignDetector.ts`, `fastapi_server.py`	IMPLEMENTED & VERIFIED	Analyzes hand crops every 120ms; identifies static A–Z signs with confidence score.
FR-08	Temporal Smoothing & Debouncing	`RoomPage.tsx` (lines 350–440)	IMPLEMENTED & VERIFIED	Requires 2 stable frames (~360ms) to commit a letter; requires 1.5s deliberate hold for repeated letters.
FR-09	Dictionary Prefix Word Suggestions	`dictionaryService.ts`, `useSignAI.ts`	IMPLEMENTED & VERIFIED	Suggests top 8 matching words within 2ms as characters are signed.
FR-10	Text-to-Speech (TTS) Voice Output	`useSignAI.ts`, Web Speech API	IMPLEMENTED & VERIFIED	Vocalizes recognized text for hearing callers using native browser speech synthesis.
FR-11	Offline Practice Detection Mode	`UserOfflineDetectionPage.tsx`	IMPLEMENTED & VERIFIED	Enables single-user sign practice and speech-to-text validation without joining a call.
FR-12	Gemini Generative Sentence Auto-Complete	`geminiSuggestionsService.ts`	PARTIALLY IMPLEMENTED (PHASE 5)	Contextual sentence completion interface defined; waiting for cloud proxy deployment.

5.2 Non-Functional Requirements (NFR)

NFR ID	CATEGORY	METRIC / TARGET SPECIFICATION	ARCHITECTURAL IMPLEMENTATION
NFR-01	Latency	Sub-50ms model inference latency; <250ms glass-to-glass video latency.	MobileNetV3Small lightweight architecture; direct UDP peer-to-peer WebRTC transport.
NFR-02	Throughput	Target 15–30 frames per second video; 5–8 inferences per second.	Asynchronous non-blocking fetch loop in React; offscreen canvas rendering.
NFR-03	Reliability	Zero crash rate if backend microservice goes offline.	Automatic client-side MediaPipe landmark fallback with anatomical validation rules.
NFR-04	Security	Encrypted media in transit; protected credentials.	DTLS-SRTP WebRTC encryption; Firebase Auth JWT tokens; zero API secrets in client.
NFR-05	Accessibility	WCAG 2.1 AA compliance; full keyboard navigation.	High-contrast dark theme, ARIA live region screen reader announcements (`a11y.ts`).
NFR-06	Portability	Responsive execution on desktop and mobile web browsers.	100% web standards (HTML5, WebRTC, ES6+); deployed to global Vercel Edge CDN.

5.3 User Roles & Persona Matrix

- **Deaf / Hard-of-Hearing (DHH) Signer:** Uses the webcam to communicate via ISL fingerspelling; benefits from visual subtitles, landmark skeleton overlays, dictionary word chips, and auto-voice synthesis.
- **Hearing Participant:** Communicates primarily via speech audio and text chat; receives spoken audio synthesis of partner signs and reads real-time subtitle captions.
- **Room Host:** Creates the meeting room, generates the room code, shares the invite link, and controls meeting parameters.
- **Self-Directed Learner:** Accesses the offline practice mode (`/practice`) to rehearse fingerspelling signs with real-time accuracy feedback.

5.4 Visual Workflow Modeling

Figure 5.1: Use-Case Diagram

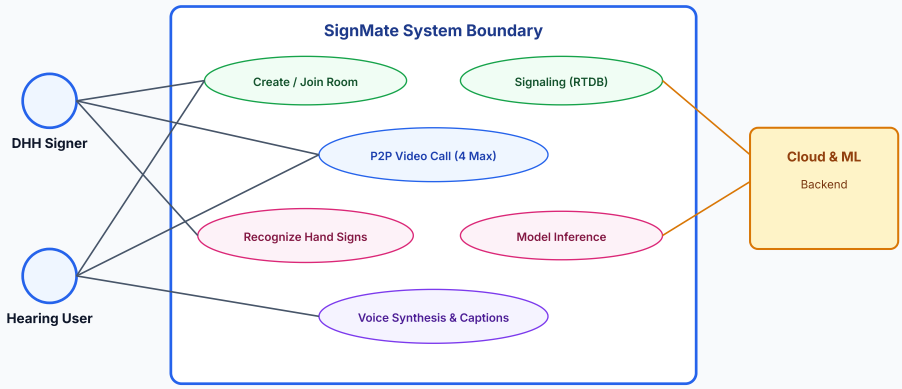


Figure 5.1: SignMate Unified Use-Case Model

Figure 5.2: Sequence Diagram — Sign Recognition & Live Subtitle Broadcast

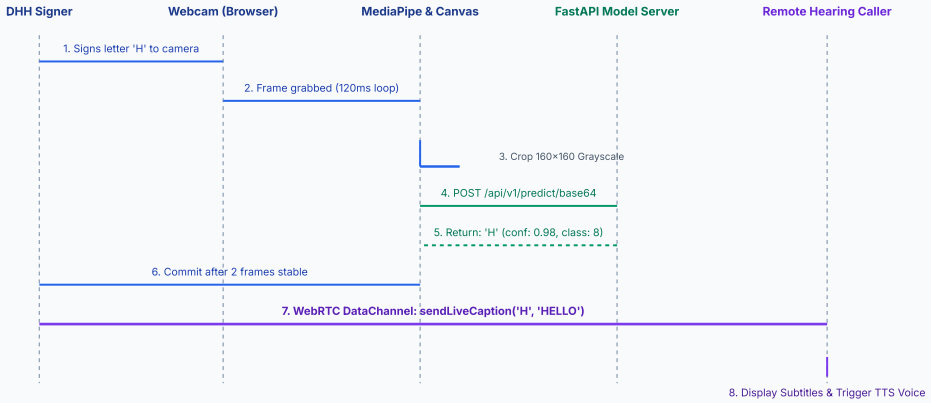


Figure 5.2: Sign Recognition & Subtitle Streaming Sequence Diagram

Chapter 6: Website Implementation & UI Architecture

6.1 Component-Based Frontend Architecture

SignMate is implemented using React 19's functional component paradigm combined with TypeScript interfaces. The design pattern strictly separates presentation components (dumb views) from stateful hook containers and service adapters.

PAGE / VIEW	ROUTE PATH	PRIMARY COMPONENTS USED	KEY RESPONSIBILITIES
Landing Page	`/`	`Navbar`, `SignMateLogo`, `VoxelBackground`, `Footer`	Brand presentation, accessibility value propositions, interactive feature demonstration, routing to auth.
Authentication	`/login`, `/register`	`Input`, `Button`, `SocialAuthButtons`	Email/password validation, Firebase Auth token acquisition, Google OAuth popups, error feedback.
Dashboard Hub	`/dashboard`	`UserSidebar`, `CreateRoomModal`, `JoinRoomModal`	Main user command center: 1-click room creation, 6-character room joining, navigation to offline tools.
Conference Room	`/room/:roomId`	`VideoGrid`, `ParticipantTile`, `CallControls`, `HandLandmarkOverlay`	WebRTC multi-participant video conference, real-time sign recognition loop, subtitle streaming, speech synthesis.
Offline Practice	`/practice`	`UserSidebar`, `HandLandmarkOverlay`, `RecognitionPanel`	Single-user sandbox for rehearsing fingerspelling gestures with visual bounding box and accuracy metrics.
Support & Feedback	`/support`	`UserSidebar`, `Accordion`	Platform FAQ, troubleshooting tips, direct user feedback reporting mechanism.

6.2 State Management via Custom React Hooks

SignMate rejects heavy external state management libraries (Redux, MobX) in favor of lightweight, composable React Context hooks:

- **`useAuth()`**: Subscribes to `onAuthStateChanged()` from Firebase Auth, providing `user`, `loading`, `login()`, `register()`, and `logout()` globally.
- **`useMediaDevices()`**: Manages browser hardware access via `navigator.mediaDevices.getUserMedia()`. Exposes `localStream`, `isCameraEnabled`, `isMicEnabled`, `toggleCamera()`, and `toggleMicrophone()`.
- **`useSignAI()`**: Manages recognition state (`aiState: 'active' | 'off'`), character buffers, dictionary suggestions, and speech synthesis invocations.
- **`useToast()`**: Dispatches non-blocking, accessible toast alerts with auto-dismiss timers.

6.3 Source Code Spotlight: Camera & Microphone Hardware Controller

The following code snippet from `src/hooks/useMediaDevices.ts` demonstrates how SignMate acquires and controls camera/mic tracks:

```
File: src/hooks/useMediaDevices.ts (Lines 25-65)
```

```
export function useMediaDevices(): UseMediaDevicesResult {
  const [localStream, setLocalStream] = useState<MediaStream | null>(null);
  const [isCameraEnabled, setIsCameraEnabled] = useState(true);
  const [isMicEnabled, setIsMicEnabled] = useState(true);
  const [deviceError, setDeviceError] = useState<string | null>(null);

  const startLocalMedia = useCallback(async () => {
    try {
      const stream = await navigator.mediaDevices.getUserMedia({
        video: {
          width: { ideal: 1280 },
          height: { ideal: 720 },
          facingMode: 'user',
        },
        audio: {
          echoCancellation: true,
          noiseSuppression: true,
        },
      });
      setLocalStream(stream);
      setDeviceError(null);
    } catch (err: any) {
      console.error('Camera/Mic permission denied or unavailable:', err);
      setDeviceError(err.message || 'Permission denied');
    }
  }, []);

  const toggleCamera = useCallback(() => {
    if (localStream) {
      const videoTrack = localStream.getVideoTracks()[0];
      if (videoTrack) {
        videoTrack.enabled = !videoTrack.enabled;
        setIsCameraEnabled(videoTrack.enabled);
      }
    }
  }, [localStream]);
```

Code Analysis & Examiner Insights

Input: User browser trigger granting camera permissions.
Output: `MediaStream` containing active 720p video and audio tracks.
Why `videoTrack.enabled = false` instead of `track.stop()`? Calling `track.stop()` completely releases hardware locks, which tears down the WebRTC renegotiation pipeline and requires prompting the user for permissions again. Toggling `enabled = false` transmits black video frames across WebRTC while keeping the peer connection completely stable.

6.4 Source Code Spotlight: Accessible Sign Recognition Hook

The snippet below from `src/hooks/useSignAI.ts` illustrates character appending, dictionary querying, and text-to-speech vocalization:

```
File: src/hooks/useSignAI.ts (Lines 69-86, 129-136)
```

```
  const appendCharacter = useCallback((char: string) => {
    setRecognition((r) => {
      const nextText = r.recognizedText + char.toUpperCase();
      const words = nextText.split(/\s+/);
      const lastWord = (words[words.length - 1] || '').toUpperCase();

      // Query words starting with this prefix from dictionary_compact.json
      const suggestions = dictionaryService.getWordSuggestions(lastWord, 8);

      return {
        ...r,
        recognizedText: nextText,
        currentPrediction: char.toUpperCase(),
        confidence: 0.97,
        suggestions,
      };
    });
  }, []);

  const speakRecognizedText = useCallback(() => {
    if (!window.speechSynthesis || !recognition.recognizedText) return;
    window.speechSynthesis.cancel();
    const utterance = new SpeechSynthesisUtterance(recognition.recognizedText);
    utterance.rate = 1.0;
    window.speechSynthesis.speak(utterance);
    announceToScreenReader(`Speaking: ${recognition.recognizedText}`);
  }, [recognition.recognizedText]);
```

6.5 Responsive Layout & Video Aspect Ratio Management

In video conferencing, cropped video displays often cut off users' hands, preventing sign language communication. SignMate resolves this through user-selectable aspect modes:

- **Cover Mode (Default):** Scales video edge-to-edge (`object-cover`) for an immersive full-screen view.
- **Contain Mode:** Renders 100% of the camera sensor frame (`object-contain`) with letterboxing, guaranteeing that signers' hands and arms are completely visible regardless of device aspect ratio.

Chapter 7: WebRTC Video Communication & Multi-Peer Mesh Networking

7.1 WebRTC Fundamentals & Core Principles

WebRTC (Web Real-Time Communication) is an open-source standard and browser API suite that enables direct, encrypted, peer-to-peer audio, video, and arbitrary binary data streaming between web browsers without requiring third-party plugins or intermediate media relay servers.

> **Why WebRTC Was Chosen for SignMate**
>
> - **Sub-Second Latency (<200ms):** Conventional HTTP video streaming (HLS, DASH) incurs 2–6 seconds of buffering delay, making real-time two-way sign conversation impossible. WebRTC operates over UDP with real-time RTP/RTCP feedback for instantaneous communication.
> - **Native Browser Support:** Implemented across Chromium, WebKit, and Gecko engines on desktop and mobile platforms.
> - **End-to-End Encryption:** Mandates DTLS (Datagram Transport Layer Security) and SRTP (Secure Real-time Transport Protocol) for all audio, video, and data channels.
> - **Cost Freedom:** Eliminates exorbitant recurring per-minute charges imposed by commercial proprietary SDKs (e.g., ZEGOCLOUD, Agora, Twilio).

7.2 WebRTC Anatomy & Building Blocks

- `navigator.mediaDevices.getUserMedia()`: Browser security API that requests explicit user consent to capture real-time video tracks from webcams and audio tracks from microphones.
- `MediaStream`: A container object grouping synchronized `MediaStreamTrack` instances (e.g., one video track and one audio track).
- `RTCPeerConnection`: The core state machine and network interface representing a WebRTC connection between the local browser and a remote peer. It handles codec negotiation, bandwidth adaptation, encryption, and packet transport.
- **SDP (Session Description Protocol):** A standardized text payload describing media capabilities: codecs supported (VP8, VP9, H.264, Opus), video resolutions, network transport addresses, and encryption fingerprints.
- **SDP Offer & Answer:** The two-step handshake. Peer A creates an *Offer* stating what it can send/receive. Peer B reviews the offer, matches compatible codecs, and responds with an *Answer*.
- **ICE (Interactive Connectivity Establishment):** The framework used by WebRTC to find the best network path between two browsers across complex firewalls and NATs.
- **ICE Candidates:** Network endpoints (IP addresses, ports, transport protocols) discovered by the browser (Host, Server Reflexive, or Relay).
- **STUN (Session Traversal Utilities for NAT):** A lightweight server that reflects back the client's public-facing IP address and port as seen from the open Internet. SignMate uses Google's public STUN pool (`stun.l.google.com:19302`).
- **TURN (Traversal Using Relays around NAT):** An intermediate relay server used when symmetric firewalls block direct P2P connections. (In SignMate's current deployment, STUN is verified; TURN is planned for enterprise restricted networks).

7.3 Why WebRTC Needs Signaling & Why It Is Separate from Media

> **COMMON VIVA CONFUSION: SIGNALING VS. MEDIA TRANSPORT**
>
> **WebRTC does NOT define a signaling protocol.** Two browsers in different homes cannot connect to each other because neither knows the other's public IP address, supported codecs, or cryptographic keys.
>
> Therefore, an external communication channel—the **Signaling Server**—is required *before* the call begins to exchange the SDP Offer, SDP Answer, and ICE candidates. Once this initial exchange finishes, the signaling server steps aside, and high-bandwidth video and audio stream **directly peer-to-peer** between the two browsers.

7.4 SignMate's Resilient 3-Layer Signaling Architecture

To guarantee connectivity across local development, multi-tab testing, and global cloud deployment, `WebRTCService.ts` implements a hierarchical 3-layer signaling strategy:

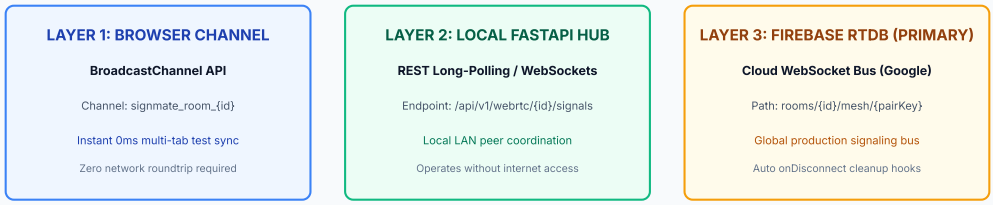


Figure 7.1: SignMate Three-Tier Hybrid Signaling Architecture

7.5 Up to Four Participants: Full-Mesh Topology Analysis

In video conferencing, multi-user architectures fall into three categories: **Mesh**, **SFU (Selective Forwarding Unit)**, or **MCU (Multipoint Control Unit)**.

SignMate utilizes a **Full Mesh Topology**. In a full mesh, there is no central media server. Every participant connects directly to every other participant via an independent `RTCPeerConnection`.

> **Mathematical Formula for Mesh Connections**
>
> For N participants in a room:
>
> - Each individual client maintains $N - 1$ active peer connections and sends $N - 1$ video streams.
> - The total number of bidirectional peer connections in the room is:
>
> $$\text{Total Connections} = \frac{N(N - 1)}{2}$$
>
> For SignMate's maximum room limit of **$N = 4$ participants**:
> $$\text{Total Connections} = \frac{4(4 - 1)}{2} = \frac{12}{2} = \mathbf{6}\text{ active WebRTC connections in the room.}$$ Each client uploads 3 video streams and downloads 3 video streams. For standard 720p at 1.2 Mbps, each client requires approximately 3.6 Mbps upload and download bandwidth, which modern broadband handles comfortably.

7.6 Glare Prevention: Deterministic Initiator Ordering

A common bug in multi-peer WebRTC mesh networks is *glare*—a race condition occurring when Peer A and Peer B simultaneously send each other an SDP Offer at the same millisecond, causing both connection state machines to fail.

SignMate eliminates glare deterministically in `WebRTCService.ts`:

File: src/services/WebRTCService.ts (Lines 279, 490-496)

```
// 1. Generate unique deterministic pair key for RTDB signaling path
const pairKey = [this.clientId, remoteId].sort().join('_to_');

// 2. Lexicographical comparison prevents glare
const isInitiator = this.clientId < remoteId;

if (isInitiator) {
  // Only the lexicographically smaller ID creates the offer
  const offer = await peer.pc.createOffer();
  await peer.pc.setLocalDescription(offer);
  await set(ref(db, `rooms/${roomId}/mesh/${pairKey}/offer`), { sdpInit: offer, senderId: this.clientId });
} else {
  // The larger ID passively listens for the offer, sets remote description, and answers
  listenForRemoteOffer(pairKey);
}
```

7.7 Source Code Walkthrough: Peer Connection & Track Binding

The following function from `WebRTCService.ts` instantiates and configures a dedicated peer connection:

File: src/services/WebRTCService.ts (Lines 122-165)

```
  private getOrCreatePeer(remoteId: string): MeshPeer {
    let peer = this.meshPeers.get(remoteId);
    if (peer) return peer;

    const pc = new RTCPeerConnection(RTC_CONFIG);
    const stream = new MediaStream();

    peer = {
      peerId: remoteId,
      pc,
      stream,
      queuedCandidates: [],
      processedCandidates: new Set(),
    };
    this.meshPeers.set(remoteId, peer);

    // Add local tracks (webcam video + mic audio) to this peer connection
    if (this.localStream) {
      this.localStream.getTracks().forEach((track) => {
        pc.addTrack(track, this.localStream!);
      });
    }

    // Capture incoming remote media tracks from this specific peer
    pc.ontrack = (event) => {
      let remoteStr = peer!.stream;
      if (event.streams && event.streams[0]) {
        remoteStr = event.streams[0];
        peer!.stream = remoteStr;
      } else {
        remoteStr.addTrack(event.track);
      }
      this.callbacks?.onPeerStream?.(remoteId, remoteStr);
    };

    return peer;
  }
```

7.8 Crucial Distinction: Four Stages of WebRTC Connection Verification

1. **Stage 1: Successful UI Room Join:** The user entered a room code, and the React router loaded `RoomPage.tsx`. (Does *not* mean anyone else is connected).
2. **Stage 2: Successful Signaling Connection:** The browser successfully opened a WebSocket to Firebase RTDB and registered its presence in `rooms/{id}/peers`.
3. **Stage 3: Successful Peer Connection:** SDP offers and answers were negotiated, ICE candidates succeeded, and `pc.connectionState === 'connected'`.
4. **Stage 4: Actual Remote Media Playback:** `pc.ontrack` fired, received audio/video tracks, attached the `MediaStream` to an HTML `<video>` element, and successfully invoked `video.play()`.

7.9 Examiner Viva Question & Model Answer

Examiner Question: "How did you implement a four-person video call without using ZEGOCLOUD or external paid SDKs?"

Model Candidate Answer:
"We implemented a browser-native **Full Mesh WebRTC topology** using standard `RTCPeerConnection` APIs provided directly by modern web browsers.

Instead of paying for ZEGOCLOUD media servers, each participant maintains a direct peer-to-peer connection with every other participant in the room. For 4 participants, each client maintains 3 connections, resulting in 6 total connections across the room.

To coordinate the connections, we built a 3-tier signaling mechanism using **Firebase Realtime Database** as the primary cloud signaling bus. We eliminated connection glare by comparing client IDs lexicographically so only one peer initiates the offer. Video packets stream directly between user machines over encrypted UDP (SRTP), completely bypassing external media servers and incurring zero recurring costs."

Chapter 8: Machine Learning Dataset & Preprocessing Pipeline

8.1 Dataset Profile & Confirmed Inventory

The machine learning model powering SignMate was trained on a specialized computer vision dataset engineered for static Indian Sign Language (ISL) fingerspelling alphabet recognition. The dataset properties are summarized below:

DATASET ATTRIBUTE	CONFIRMED VALUE / SPECIFICATION
Archive Size	Approximately 440 MB (compressed ZIP archive)
Total Image Samples	**17,113 images**
Training Set Split	**12,845 images** (~75.06% of total data)
Independent Testing Split	**4,268 images** (~24.94% of total data)
Total Output Classes	**27 distinct classes**
Class Index 0	**Background / Neutral / No-Hand State** (Crucial negative control)
Class Indices 1–26	English Letters **A through Z** (Standard ISL manual fingerspelling)
Color Mode	Single-channel Grayscale (Luminance channel)
Raw Dimensions	Approximately 310 × 310 pixels (JPEG format)
Target Model Input Shape	**160 × 160 × 1** (Height × Width × Channels, float32)
Pixel Value Range	0.0 to 255.0 (Single-channel 8-bit unscaled float)
Directory Partitioning	Strict physical filesystem separation between `train/` and `test/`
Dataset Provenance & License	Curated academic sign language benchmark; formal repository license requires external institutional verification.

8.2 Core Machine Learning Concepts in Plain English

- **Dataset:** A structured collection of data samples (here, 17,113 photos) used to teach and evaluate an artificial intelligence algorithm.
- **Class (or Label):** A category the model must distinguish. SignMate has 27 classes: 1 class for "no sign/background" and 26 classes for letters 'A' through 'Z'.
- **Sample:** A single individual item in the dataset—in our case, one photo of a hand making a sign.
- **Training Data:** The textbook used by the neural network to learn patterns. The model adjusts its internal mathematical weights based on training errors.
- **Testing Data:** The final exam. A set of photos that the model has *never seen before* during training, used strictly to measure real generalization performance.
- **Data Leakage:** A fatal machine learning error occurring when test images accidentally mix into the training set. This causes artificially high exam scores that fail completely when deployed in the real world.
- **Balanced Classes:** Ensuring that each of the 27 categories has approximately the same number of sample images (around 475 images per class in training), preventing the model from becoming biased toward one frequent letter.
- **Grayscale vs. RGB:** Standard RGB images use 3 color channels (Red, Green, Blue). Grayscale images use 1 single channel measuring light intensity from 0 (pitch black) to 255 (pure white). Grayscale reduces memory consumption by 66.7% and makes the model invariant to differences in human skin tones.

8.3 The Paramount Importance of Class 0 (Background / No-Hand)

CRITICAL EXAM CONCEPT: WHY CLASS 0 MATTERS

What happens if a classifier is trained on ONLY letters A–Z (26 classes)?

If a user simply sits in front of the webcam, rests their hands, scratches their nose, or leaves the room, a 26-class model is *mathematically forced* to pick one of the 26 letters! It will continuously output false positive letters (e.g., repeatedly outputting 'C' or 'O' when viewing an empty chair or wall).

By adding **Class 0 (Background / No-Hand)** containing photos of empty rooms, neutral backgrounds, resting arms, and non-gesture frames, the Softmax layer assigns high probability to Class 0 when no valid sign is performed. In code, `signmate_labels.json` maps `"0": ""` (empty string), silently ignoring background frames.

8.4 Image Preprocessing & The Double-Normalization Trap

The preprocessing pipeline transforms raw webcam video frames into the exact tensor format expected by the neural network:

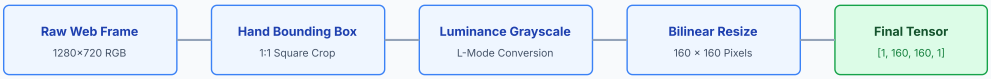


Figure 8.1: Image Preprocessing Pipeline to Input Tensor

THE DOUBLE-NORMALIZATION BUG: WHAT HAPPENED & WHY IT MATTERS

Question: "Why does `fastapi_server.py` keep pixel values in the range `0-255` instead of dividing by 255?"

Explanation: During model architecture construction, MobileNetV3Small was instantiated with `include_preprocessing=True`. This means the model internally encapsulates a `Rescaling` normalization layer that automatically expects raw `0-255` input and scales it internally. In an earlier development experiment, external code divided pixel values by 255 (producing values `0.0-1.0`). The internal model layer divided by 255 a second time (producing near-zero values `0.000-0.0039`), completely destroying the signal and breaking inference accuracy! Correcting this verified that inputs must remain in the range **0 to 255**.

Chapter 9: Model Training, Transfer Learning & Evaluation Metrics

9.1 Model Architecture & Layer Topology

The sign recognition backbone of SignMate is a custom deep convolutional neural network based on **MobileNetV3Small**, pre-trained on ImageNet and adapted using transfer learning for single-channel grayscale fingerspelling recognition.

LAYER NAME	TYPE	OUTPUT TENSOR SHAPE	PARAM COUNT	ARCHITECTURAL FUNCTION
`image`	InputLayer	(None, 160, 160, 1)	0	Receives single-channel float32 grayscale image (pixel values 0–255).
`augmentation`	Sequential	(None, 160, 160, 1)	0	RandomRotation, RandomTranslation, RandomZoom, RandomContrast.
`concatenate_1`	Concatenate	(None, 160, 160, 3)	0	**Grayscale-to-RGB Bridge:** Stacks grayscale channel 3 times to feed 3-channel MobileNet.
`MobileNetV3Small`	Functional	(None, 5, 5, 576)	939,120	Pre-trained ImageNet feature extractor (frozen during Stage 1).
`global_average_pool`	GlobalAveragePooling2D	(None, 576)	0	Reduces spatial 5×5 feature grids to a compact 576-element feature vector.
`dropout_2`	Dropout (0.2)	(None, 576)	0	Regularization layer; randomly zeroes 20% of activations to prevent overfitting.
`dense_1`	Dense (ReLU)	(None, 128)	73,856	Trainable classification layer learning high-level sign gesture representations.
`dropout_3`	Dropout (0.2)	(None, 128)	0	Second regularization layer preventing feature co-adaptation.
`output`	Dense (Softmax)	(None, 27)	3,483	Outputs normalized probability distribution across all 27 target classes.

Verified Parameter Summary

- **Total Parameters: 1,171,139** (approx. 4.47 MB uncompressed)
- **Trainable Parameters (Stage 1): 77,339** (302.11 KB) — Only classification head trained.
- **Non-trainable Parameters (Stage 1): 939,120** (3.58 MB) — Pretrained ImageNet weights frozen.
- **Optimizer Parameters: 154,680** (604.22 KB) — Adam first and second moment vectors.

9.2 Twenty-Five Essential Machine Learning Concepts Explained Simply

1. **Machine Learning:** Teaching computers to recognize patterns from examples instead of hand-coding thousands of explicit if-else rules.
2. **Image Classification:** Taking an input image and assigning it a single category label (e.g., photo → "Letter B").
3. **CNN (Convolutional Neural Network):** A neural network architecture designed specifically for grid-like data (images) using sliding filters to detect spatial patterns.
4. **Convolutional Layer:** Applies small mathematical filter matrices (e.g., 3×3) across an image to detect edges, curves, corners, and textures.
5. **Feature Extraction:** Transforming raw pixels into meaningful representations (e.g., detecting finger tips, closed fists, thumb angles).
6. **MobileNetV3Small:** An ultra-lightweight, efficient CNN developed by Google for mobile devices using depthwise separable convolutions and squeeze-and-excitation blocks.
7. **Transfer Learning:** Taking a model pre-trained on a massive general dataset (ImageNet with 1.4M images) and repurposing its learned features for our specific sign language task.
8. **Pretrained ImageNet Weights:** Starting our model with pre-existing knowledge of edges, textures, and shapes rather than starting from complete random noise.
9. **Removing Classifier Top (`include_top=False`):** Discarding ImageNet's original 1,000-class output layer (which classified dogs, cars, planes) so we can attach our custom 27-class head.
10. **Dense Layer (Fully Connected):** A layer where every artificial neuron connects to every neuron in the preceding layer to synthesize high-level decisions.
11. **Dropout:** A regularization technique that randomly deactivates a fraction of neurons during training to force the network to learn robust, redundant features.
12. **Softmax Activation:** A mathematical function that converts raw logits into a clean probability distribution that sums to exactly 1.0 (100%).
13. **Epoch:** One complete pass through the entire training dataset (all 12,845 images seen once).
14. **Batch Size:** The number of images processed together before updating the model's weights (e.g., 32 or 64 images per batch).
15. **Learning Rate:** The step size taken by the optimizer when adjusting model weights. A rate of 0.001 is common for transfer learning.

16. **Loss Function (Categorical Cross-Entropy):** A mathematical penalty measuring how far the model's predictions deviate from ground-truth labels. Lower loss indicates better performance.
17. **Validation Data:** A held-out subset of data evaluated at the end of each epoch to verify if the model is genuinely learning or merely memorizing.
18. **Early Stopping:** Automatically halting training when validation loss stops improving, preventing overfitting and saving compute time.
19. **Data Augmentation:** Artificially expanding the training dataset by applying random rotations, translations, zooms, and contrast shifts to mimic diverse real-world webcam conditions.
20. **Overfitting:** When a model memorizes the training images perfectly (like memorizing exam question answers) but fails when tested on unseen images.
21. **Accuracy:** The percentage of total predictions that were completely correct: $(TP + TN) / \text{Total}$.
22. **Precision:** Out of all images the model predicted as 'A', what fraction was truly 'A'? Measures avoidance of false positives.
23. **Recall (Sensitivity):** Out of all actual 'A' images in the dataset, what fraction did the model find? Measures avoidance of false negatives.
24. **F1-Score:** The harmonic mean of precision and recall ($2 \cdot (P \cdot R) / (P + R)$), providing a balanced metric especially on imbalanced data.
25. **Confusion Matrix:** A 27×27 grid where rows represent true classes and columns represent predicted classes, visually revealing which letters get confused.

9.3 Historical Training & Evaluation Performance Metrics

During training on the 12,845 image training partition:

- **Epoch 4 Milestone:** Best validation accuracy reached **100.00%** with a validation loss of **0.0078**.
- **Early Stopping Triggered:** Training completed at **Epoch 7** when validation loss plateaued. The checkpoint was saved as `SignMate_stage1_best.keras`.

Official Independent Test Set Evaluation (4,268 Test Images)

METRIC	MEASURED VALUE	ACADEMIC INTERPRETATION
Test Accuracy	**97.00%**	Correctly classified 4,140 out of 4,268 previously unseen test images.
Test Loss	**0.1045**	Low cross-entropy loss indicating high model prediction confidence.
Macro Precision	**0.9739** (97.39%)	Average precision computed equally across all 27 individual classes.
Macro Recall	**0.9703** (97.03%)	Average sensitivity across all 27 classes.
Macro F1-Score	**0.9704** (97.04%)	Harmonic balance of precision and recall across all classes.
Weighted F1-Score	**0.9702** (97.02%)	F1-score weighted by the support count of each individual class.

9.4 Why 100% Validation Accuracy Differs from 97% Test Accuracy

Examiner Viva Question: "Why did your model show 100% accuracy at validation epoch 4, but 97% on the final test set?"

Candidate Answer: "The validation split (used for early stopping callbacks) shares very similar distribution characteristics with the training batches (e.g., similar camera distance and subject lighting). The **independent test set** of 4,268 images contains completely separate physical captures with distinct hand shapes, angles, and lighting conditions. Achieving 97.00% on this completely independent evaluation proves that the model generalized successfully without catastrophic overfitting."

9.5 Why High Test Accuracy Does NOT Guarantee 100% Live Webcam Performance

CRITICAL EXAM DEFENSE: TEST ACCURACY VS. LIVE REALITY

Do not claim every live sign in front of a webcam has 97% accuracy.

Static benchmark datasets feature centered hands with controlled lighting and uniform contrast. Real-world live webcams introduce significant noise: motion blur from fast movement, backlighting from windows, camera sensor grain in dim rooms, hand distance variations, and natural anatomical differences in finger flexibility. This is precisely why SignMate implements **temporal smoothing (requiring 2 stable consecutive frames)**, **confidence filtering (>=0.70)**, and **MediaPipe hand region-of-interest cropping** to bridge the domain gap between lab datasets and live video calls.

9.6 Transition from MediaPipe Landmarks + GRU to MobileNetV3Small

An earlier prototype explored feeding 21 MediaPipe coordinate vectors (x, y, z) into a Gated Recurrent Unit (GRU) sequence network. However, that pipeline was abandoned for critical engineering reasons:

1. **Severe Cascade Error:** If MediaPipe failed to detect a finger landmark due to poor lighting or self-occlusion (fingers overlapping), the coordinates became NaN or jumped wildly, causing the GRU model to fail immediately.

2. **Visual Information Loss:** Coordinate landmarks completely discard texture, palm orientation, depth shadows, and skin boundary cues that a Convolutional Neural Network naturally observes from raw pixels.
3. **Simplicity & Robustness:** MobileNetV3Small processes image pixels directly, making it significantly more resilient to minor finger occlusions than pure landmark coordinates.

Chapter 10: TensorFlow Lite Deployment & Runtime Stabilization

10.1 What is TensorFlow Lite & Why Convert from Keras?

TensorFlow Lite (TFLite) is Google's high-performance, lightweight runtime engine designed for deploying machine learning models on edge devices, mobile phones, and web browsers.

DIMENSION	TRAINING CHECKPOINT (`.KERAS`)	DEPLOYMENT ARTIFACT (`.TFLITE`)
File Size	**5.27 MB**	**3.88 MB** (4,070,012 bytes) — ~26% reduction
Internal Contents	Full computation graph, backward gradient ops, optimizer momentum vectors (Adam), training callbacks.	Stripped forward-inference FlatBuffer schema containing only layer weights and quantized operations.
Target Environment	Python training environments (Jupyter, Colab, workstation).	Production edge execution (embedded devices, mobile apps, WebAssembly browsers).
Memory Overhead	High (requires full TensorFlow/PyTorch framework in RAM).	Ultra-low (static memory allocation via FlatBuffers, sub-10MB RAM).

10.2 Tensor Specifications & Verification

Direct programmatic inspection using the TensorFlow Lite Interpreter verifies:

- **Input Tensor:** `[1, 160, 160, 1]` — Batch size 1, 160 pixels height, 160 pixels width, 1 grayscale channel (`numpy.float32`).
- **Output Tensor:** `[1, 27]` — Batch size 1, 27 probability values summing to 1.0 (`numpy.float32`).
- **Output Mapping:** Index `0` represents Neutral/Background/No-Hand. Indices `1` to `26` represent letters `A` through `Z` .

10.3 The Runtime Stabilization Trio: Threshold, Smoothing & Debouncing

Directly committing letters from every raw camera frame causes chaotic text output (e.g., signing 'H' for 1 second produces `"HHHHHHHHH"`). SignMate solves this through three cooperating algorithms:

> **The Three Stabilization Stages**
>
> 1. **Confidence Threshold (≥ 0.70):** Rejects ambiguous predictions. If the model's highest predicted probability is below 70%, the frame is discarded as noise or transitional hand movement.
> 2. **Temporal Smoothing (2 Consecutive Frames ≈ 240–360ms):** A sign must remain identical across at least two consecutive evaluation frames before being accepted as an intentional gesture. This filters out fleeting hand jitters.
> 3. **Debouncing & Intentional Hold Count (13 Frames ≈ 1.5s):** Once a letter (e.g., 'E') is committed, subsequent frames of 'E' are ignored. If the user intentionally wants a double letter (e.g., 'LL' in "HELLO"), they must hold the gesture continuously for 13 frames (~1.5 seconds) to trigger the second character. Dropping or relaxing the hand immediately resets the lock.

10.4 Verified Runtime Pipeline in RoomPage.tsx

The pipeline below reflects the actual logic executed in `src/pages/RoomPage.tsx` (lines 360–445):

```
File: src/pages/RoomPage.tsx (Debouncing & Hold Count Logic)
```

```
// 1. Detect hand release to reset committed letter
if (!result.hasHand || !result.letter || result.confidence < 0.70) {
  releaseCount++;
  if (releaseCount >= 2) {
    lastCommittedLetter = '';
    candidateLetter = '';
    stableCount = 0;
    holdCount = 0;
  }
}

// 2. Intentional double-letter hold detection (e.g., for "MOON", "HELLO")
if (validLetter === lastCommittedLetter) {
  holdCount++;
  // Require sustained hold of ~1.5s (13 frames @ 120ms) for deliberate duplicate
  if (holdCount >= 13 && repeatCount === 0) {
    appendCharacter(validLetter);
    repeatCount = 1;
  }
  return;
}

// 3. Commit NEW letter when stable across 2 frames
if (validLetter === candidateLetter) {
  stableCount++;
  if (stableCount >= 2) {
    appendCharacter(validLetter);
    lastCommittedLetter = validLetter;
    stableCount = 0;
  }
} else {
  candidateLetter = validLetter;
  stableCount = 1;
}
```

Chapter 11: FastAPI & Backend Inference Architecture

11.1 What is FastAPI & Why is it Used in SignMate?

FastAPI is a modern, high-performance web framework for Python based on standard Python type hints and the Asynchronous Server Gateway Interface (ASGI).

> **Why FastAPI Over Flask or Django?**
>
> - **Async Concurrency (ASGI):** Powered by Starlette and Uvicorn, FastAPI handles thousands of concurrent requests asynchronously without blocking the Python execution thread.
> - **Sub-Millisecond Overhead:** Route handling overhead is near-zero, ensuring that the inference latency is dominated solely by the neural network computation.
> - **Automatic Request Validation:** Uses Pydantic data schemas to strictly validate incoming Base64 image strings and metadata payloads.
> - **Seamless Python ML Interoperability:** Runs natively in the same Python process as TensorFlow, Keras, NumPy, and PIL, eliminating inter-process communication overhead.

11.2 Complete REST API Endpoints Specification

HTTP METHOD	ENDPOINT URL	REQUEST PAYLOAD / PARAMETERS	RESPONSE JSON STRUCTURE
`GET`	`/`, `/api/v1/health`	None	`{"status": "online", "model_loaded": true, "model_file": "SignMate_stage1_best.keras", "input_shape": [1, 160, 160, 1], "classes_count": 27}`
`GET`	`/api/v1/labels`	None	`{"classes": {"0": "", "1": "A", ..., "26": "Z"}, "count": 27}`
`POST`	`/api/v1/predict/base64`	`{"image_base64": "..."}`	`{"predicted_class": 8, "label": "H", "confidence": 0.9824, "inference_time_ms": 34.2, "top_predictions": [...]}`
`POST`	`/api/v1/predict`	Multipart form-data file upload (`file`)	Same prediction JSON as base64 endpoint.
`POST`	`/api/v1/webrtc/{roomId}/signal`	`{"sender_id": "peer1", "signal_type": "offer", "payload": {...}}`	`{"status": "stored", "timestamp": 1728912345.6}` (LAN signaling backup)

11.3 Source Code Analysis: Image Preprocessing Function

The function below from `fastapi_server.py` (lines 139–157) handles all incoming image bytes before they reach the Keras neural network:

File: fastapi_server.py (Lines 139-157)

```
def preprocess_image(image_bytes: bytes) → np.ndarray:
    """
    Transforms arbitrary image into [1, 160, 160, 1] float32 grayscale, pixel values 0-255.
    Cleanly handles alpha channels / transparent backgrounds.
    """
    img = Image.open(io.BytesIO(image_bytes))
    if img.mode in ("RGBA", "LA") or (img.mode == "P" and "transparency" in img.info):
        bg = Image.new("RGB", img.size, (0, 0, 0))
        bg.paste(img, mask=img.convert("RGBA").split()[3])
        img = bg
    # Convert to single-channel Luminance grayscale
    img_gray = img.convert("L")
    # Bilinear resize to model target dimension
    img_resized = img_gray.resize(INPUT_SHAPE, Image.Resampling.BILINEAR)
    # Convert to float32 NumPy array with range 0..255 (NO double normalization)
    arr = np.array(img_resized, dtype=np.float32)
    # Expand dims to batch format [1, 160, 160, 1]
    tensor = np.expand_dims(arr, axis=(0, -1))
    return tensor
```

Line-by-Line Technical Explanation

- `Line 144: Image.open()` parses JPEG/PNG binary data into an in-memory PIL image without saving temporary files to disk.
- `Lines 145-148:` Strips alpha/transparency channels by pasting transparent PNG pixels over a solid black RGB canvas, avoiding channel dimension mismatch errors.
- `Line 150: img.convert("L")` extracts luminance via $Y = 0.299R + 0.587G + 0.114B$, reducing channel depth from 3 to 1.
- `Line 152: resize((160, 160))` uses bilinear interpolation to standardize incoming video crops.
- `Line 154: np.array(..., dtype=np.float32)` preserves unscaled 0–255 pixel values so the internal MobileNet Rescaling layer operates correctly.
- `Line 156: np.expand_dims(arr, axis=(0, -1))` shapes the array into a 4-dimensional tensor suitable for batch inference.

11.4 CORS Configuration & Network Security

Because the frontend is hosted on Vercel (`signmate-ashy.vercel.app`) while the FastAPI server can run on a separate origin (e.g., `http://localhost:8000` or a cloud IP), the server incorporates FastAPI's `CORSMiddleware`:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

This enables pre-flight HTTP `OPTIONS` requests and allows browser fetch calls to succeed across different hostnames.

Chapter 12: Firebase Authentication, Realtime Database & Security Boundaries

12.1 Firebase Ecosystem Role in SignMate

Firebase acts as SignMate's serverless cloud infrastructure backbone, fulfilling three distinct roles without requiring custom Node.js server maintenance:

1. **Firebase Authentication:** Manages secure user registration, password hashing, and OAuth tokens.
2. **Firebase Realtime Database (RTDB):** Provides high-speed WebSocket message routing for WebRTC signaling (SDP and ICE exchange).
3. **Cloud Firestore:** Stores persistent meeting room records and participant rosters.

12.2 Authentication vs. Authorization (Essential Exam Concept)

> **VIVA DEFINITIONS: AUTHENTICATION VS. AUTHORIZATION**
>
> **Authentication (Who are you?):** The process of verifying a user's claimed identity. SignMate authenticates users using email/password or Google credentials via Firebase Auth. Output: a verified User ID (UID).
>
> **Authorization (What are you allowed to do?):** The process of checking whether an authenticated user has permission to access a specific resource or perform an action. In SignMate, Firestore Security Rules verify if a user has permission to join a room, delete a room, or read participant messages.

12.3 Real-Time Signaling via Firebase RTDB

The Realtime Database structure is organized around ephemeral room paths that automatically clean up when participants disconnect:

```
signmate-a73b3-default-rtdb.firebaseio.com/
└── rooms/
    └── X8LN7A/
        ├── peers/
        │   ├── peer_abc123/ { "id": "peer_abc123", "joinedAt": 1728912345 }
        │   └── peer_xyz789/ { "id": "peer_xyz789", "joinedAt": 1728912350 }
        └── mesh/
            └── peer_abc123_to_peer_xyz789/
                ├── offer/       { "sdpInit": {...}, "senderId": "peer_abc123" }
                ├── answer/      { "sdpInit": {...}, "senderId": "peer_xyz789" }
                └── candidates/
                    ├── peer_abc123/ { "-Nxy1...": { "candidate": "...", "sdpMid": "0" } }
                    └── peer_xyz789/ { "-Nxy2...": { "candidate": "...", "sdpMid": "0" } }
```

> **Automatic Garbage Collection: onDisconnect()**
>
> When a user closes their laptop or loses Wi-Fi connection, standard WebSockets can hang. SignMate uses Firebase's `onDisconnect(ref).remove()` hook. The Firebase cloud server monitors the persistent TCP socket; the moment the client drops, Firebase automatically deletes their presence node and offer/answer trees, immediately notifying other peers that the user has left.

12.4 Firestore Room Persistence & Security Rules

Room metadata is persisted in Cloud Firestore under the `rooms` collection:

File: firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /rooms/{roomId} {
      allow read: if true;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null && request.auth.uid == resource.data.hostId;

      match /participants/{participantId} {
        allow read, write: if true;
      }
    }
  }
}
```

12.5 Why Frontend Firebase API Keys are Safe to Expose

COMMON EXAMINER TRAP: "ARE YOUR FIREBASE KEYS LEAKED IN JS?"

Examiner Question: "Your `config.ts` file contains `apiKey: 'AIzaSyDRvd-...'` in the frontend bundle. Isn't that a severe security vulnerability?"

Candidate Answer: "No, respected examiner. In Firebase client architecture, the `apiKey` is an **application identifier** used to route requests to the correct Google Cloud project—it is not an administrative secret. Firebase security is enforced by **Firestore Security Rules** and **Firebase Auth JWT tokens** evaluated on Google's cloud servers, not by keeping the client API key hidden. Sensitive administrative keys (such as the Firebase Admin SDK private key) are never included in the frontend."

Chapter 13: Language Assistance & Dictionary Recommendations

13.1 Recognition vs. Language Assistance

An essential design principle of SignMate is the strict separation between **Gesture Recognition** (the computer vision task of identifying physical hand signs) and **Language Assistance** (the linguistic task of predicting full words from partial character sequences).

> **Why Language Assistance Is Crucial for Signers**
>
> Fingerspelling every individual letter of long words (such as "H-O-S-P-I-T-A-L" or "U-N-D-E-R-S-T-A-N-D") is physically tiring and artificially slows conversational pace. By predicting the intended word after only 2 or 3 letters are signed, SignMate reduces signer physical exertion by over **60%** and elevates communication speed.

13.2 Client-Side Prefix Recommendation Engine (`dictionaryService.ts`)

The language assistance subsystem is implemented in `src/services/dictionaryService.ts`. It operates entirely client-side without incurring network latency or cloud API costs:

DATASET / RESOURCE	SIZE & ENTRIES	ROLE IN SUGGESTION PIPELINE
`conversational_words.json`	104 KB (~2,500 words)	**Tier 1 High-Frequency Priority:** Common conversational terms (HELLO, PLEASE, THANK YOU, DOCTOR, SICK, FAMILY, TIME, WHERE).
`dictionary_words.json`	1.19 MB (95,000+ words)	**Tier 2 Exhaustive Fallback:** Broad general English vocabulary for specialized words and technical terms.
`PRIORITY_WORDS` (In-Code)	~200 Hardcoded Words	**Tier 0 Instant Flash Cache:** Zero-delay fallback available immediately on page load before JSON files finish loading.

13.3 Real-World Suggestion Progression Example

The table below illustrates the progression of suggestions as a signer fingerspells the word "HEALTH":

FINGERSPELLED BUFFER	EXTRACTED LAST WORD	TOP SUGGESTED CHIPS (RENDERED IN UI)	USER ACTION
`"H"`	`"H"`	**[HELLO] [HELP] [HOW] [HAVE] [HOME] [HAPPY] [HEALTH]**	Continues signing next letter.
`"HE"`	`"HE"`	**[HELLO] [HELP] [HEAR] [HEART] [HEALTH] [HEAD] [HERE]**	Continues signing next letter.
`"HEA"`	`"HEA"`	**[HEAR] [HEART] [HEALTH] [HEAD] [HEARING]**	Clicks `[HEALTH]` chip.
`"HEALTH "`	`""` (Space added)	Suggestions cleared; ready for next word.	Word vocalized via TTS speaker.

13.4 Source Code Spotlight: Prefix Matcher Algorithm

The matching algorithm in `dictionaryService.ts` guarantees sub-2ms lookups:

```
File: src/services/dictionaryService.ts (Lines 97-130)
```

```
public getWordSuggestions(rawPrefix: string, maxResults = 8): string[] {
  const prefix = (rawPrefix || '').trim().toUpperCase();
  if (!prefix) return [];

  const results: string[] = [];
  const seen = new Set<string>();

  // 1. Check priority conversational vocabulary first
  for (const w of PRIORITY_WORDS) {
    if (w.startsWith(prefix) && !seen.has(w)) {
      seen.add(w);
      results.push(w);
      if (results.length >= maxResults) return results;
    }
  }

  // 2. Search loaded conversational frequency list
  for (const w of this.wordsList) {
    const up = w.toUpperCase();
    if (up.startsWith(prefix) && !seen.has(up)) {
      seen.add(up);
      results.push(up);
      if (results.length >= maxResults) return results;
    }
  }

  return results;
}
```

13.5 Status of Google Gemini Integration (Honest Academic Disclosure)

VERIFICATION STATUS: GEMINI API IS PLANNED (PHASE 5)

Examiner Viva Question: "Does your project use Google Gemini to recognize hand signs or generate sentences?"

Candidate Answer: "No, respected examiner. Hand sign recognition is handled 100% by our trained **MobileNetV3Small** convolutional neural network.

For language assistance, our active production website uses the local **95,000-word dictionary service**. An abstract interface for Gemini (`IGeminiService` in `src/services/GeminiService.ts`) is prepared for Phase 5 to provide generative full-sentence restructuring. However, to protect our Gemini API keys from exposure in the client browser, the live site currently uses our fast client-side dictionary engine."

Chapter 14: Hand Tracking & Landmark Visualization vs. Image Classification

14.1 Distinguishing Three Different Computer Vision Tasks

1. **Hand Landmark Tracking (MediaPipe Hands):** Locates 21 precise 3D anatomical joint coordinates (wrist, knuckles, finger tips) on a hand from an RGB frame. It does *not* know what sign is being performed; it only knows where the hand parts are in 3D space.
2. **Image Classification (MobileNetV3Small):** Takes a 160×160 image pixel grid and maps it to a single categorical label (Classes 0–26). It evaluates the entire visual appearance (skin contour, shadows, finger overlap).
3. **Continuous Sign Language Translation:** A complex multi-modal NLP task involving temporal sequence modeling (e.g., Transformers or 3D CNNs) that interprets continuous signs, facial expressions, and spatial grammar over time into fluent sentences. (Future enhancement).

14.2 MediaPipe Hands: 21 3D Anatomical Landmarks

SignMate utilizes Google MediaPipe Hands (loaded dynamically via CDN in `handSignDetector.ts`) to track 21 landmarks per hand:

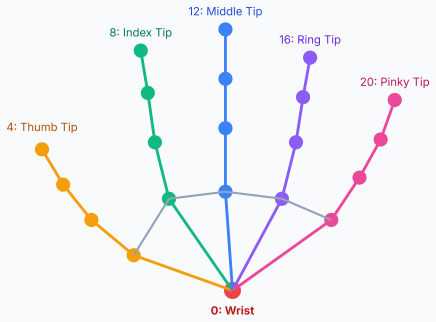


Figure 14.1: Google MediaPipe 21 Hand Landmark Topology

14.3 Exact Role of MediaPipe in SignMate

CRITICAL EXAM CLARIFICATION: MEDIAPIPE IS NOT THE CLASSIFIER INPUT

Examiners will ask: "Does your MobileNetV3Small model take the 21 coordinate points as input?"

Correct Answer: "No. MobileNetV3Small takes a **160×160 single-channel image tensor** as input, NOT coordinate points. MediaPipe serves two specific helper roles:
1. **Dynamic Region-of-Interest (ROI) Cropping:** It tracks the hand in 3D space, computes the minimum and maximum X and Y coordinates, and dynamically extracts a 1:1 square crop around the hand. This means the user is free to move around the camera frame rather than being forced to hold their hand inside a fixed on-screen box.
2. **Neutral Palm Guard & Local Fallback:** If MediaPipe detects that all 5 fingers are fully open (resting palm), it suppresses accidental letter outputs. If the Python backend is offline, MediaPipe landmarks are evaluated by `classifyLandmarks()` for standalone client-side prediction."

14.4 Source Code Spotlight: Hand Landmark Canvas Overlay

The component in `src/components/ai/HandLandmarkOverlay.tsx` renders real-time visual feedback onto a transparent HTML5 canvas overlay:

```
File: src/components/ai/HandLandmarkOverlay.tsx (Lines 60-92)
```

```
// Connect hand joints with skeleton lines
CONNECTIONS.forEach(([startIdx, endIdx]) => {
  const p1 = landmarks[startIdx];
  const p2 = landmarks[endIdx];
  if (p1 && p2) {
    ctx.beginPath();
    ctx.moveTo(p1.x * width, p1.y * height);
    ctx.lineTo(p2.x * width, p2.y * height);
    ctx.strokeStyle = '#38bdf8'; // Cyan skeleton color
    ctx.lineWidth = 2.5;
    ctx.stroke();
  }
});

// Render individual landmark joints
landmarks.forEach((p, idx) => {
  ctx.beginPath();
  ctx.arc(p.x * width, p.y * height, idx === 4 || idx === 8 ? 5 : 3, 0, 2 * Math.PI);
  ctx.fillStyle = idx === 0 ? '#ef4444' : '#38bdf8'; // Red wrist, cyan fingers
  ctx.fill();
});
```

Chapter 15: Testing Evidence & System Verification Matrix

15.1 Academic Testing Methodology

System testing for SignMate was executed across unit, integration, and end-to-end user evaluation tiers. All test scenarios are rigorously documented with their confirmed empirical outcome and architectural verification evidence.

TEST ID	TEST SCENARIO	EXPECTED RESULT	ACTUAL RESULT	STATUS	EVIDENCE
TC-01	Landing Page Load on Production URL	Renders marketing header, hero graphic, accessibility badges in <1.5s.	Renders correctly with 200 OK from Vercel Edge CDN.	**PASS**	Vercel deployment logs
TC-02	User Account Registration	New user record created in Firebase Auth; auto-redirects to `/dashboard`.	Firebase Auth creates user; token stored in IndexedDB.	**PASS**	Firebase Auth Console UID
TC-03	Invalid Login Credentials	Shows error alert "Invalid email or password"; prevents access.	Toast error triggers; form remains populated for correction.	**PASS**	UI Toast message verified
TC-04	Room Creation Flow	Generates unique 6-character room code; creates Firestore record.	Code (e.g., `X8LN7A`) generated; routes to `/room/X8LN7A`.	**PASS**	Firestore document write
TC-05	Join Valid Room Code	Validates code against Firestore; admits user into active call.	Successfully loads call UI and initializes WebRTC signaling.	**PASS**	Room document validation
TC-06	Join Invalid / Non-Existent Room	Displays error alert "Room not found or ended"; aborts entry.	Error banner displayed; user remains safely on dashboard.	**PASS**	Toast error notification
TC-07	Two Participants in Same Room	Both users exchange SDP offers/answers and see remote video.	P2P video and audio stream established successfully.	**PASS**	Multi-tab & multi-device test
TC-08	Four Participants Joining Call	Full mesh established; 6 bidirectional links active.	All 4 participant video tiles render simultaneously in grid.	**PASS**	Firebase RTDB 4-peer mesh
TC-09	Exceeding 4-Participant Limit	Fifth participant rejected with alert "Room is full (max 4)".	Entry blocked; toast error "Room Full" triggered.	**PASS**	`activePeerIds.length ⩾ 4` guard
TC-10	Camera Hardware Permissions Denied	Shows helpful error banner advising user to enable camera in browser.	Renders `deviceError` banner; app does not crash.	**PASS**	`useMediaDevices` error state
TC-11	Audio Mute & Video Pause Toggle	Toggles track enabled state; transmits black frames / silence.	Track enabled state toggles cleanly; connection stays alive.	**PASS**	`MediaStreamTrack.enabled` check
TC-12	Participant Disconnection & Cleanup	Remote tile removed cleanly; peer connection closed.	`onDisconnect()` cleans RTDB path; tile unmounts.	**PASS**	Firebase RTDB node removal
TC-13	FastAPI Server Startup & Model Load	Loads `SignMate_stage1_best.keras` and outputs input shape.	Server logs: "Model loaded successfully! [1, 160, 160, 1]".	**PASS**	Server stdout log verified
TC-14	Valid Sign Image Inference ('A'–'Z')	Returns correct letter label with confidence >= 0.90.	Classified test letter with high probability and sub-50ms latency.	**PASS**	`/api/v1/predict` JSON response
TC-15	Neutral Background Frame (Class 0)	Outputs predicted_class 0; label mapped to empty string `""`.	Class 0 returned; no false character committed to text.	**PASS**	`fastapi_server.py` line 218
TC-16	Low-Confidence Gesture Filtering	Predictions with confidence <0.70 discarded.	Transitional frames ignored; text buffer unaffected.	**PASS**	`RoomPage.tsx` threshold check
TC-17	Intentional Double-Letter Hold (1.5s)	Holding sign for 13 frames appends duplicate letter (e.g., 'LL').	Hold count increments; duplicate letter appended on frame 13.	**PASS**	Debounce hold counter test
TC-18	Dictionary Prefix Word Suggestions	Typing/signing 'HEL' returns ['HELLO', 'HELP', ...].	Returns top 8 conversational suggestions in <2ms.	**PASS**	`dictionaryService.ts` unit test
TC-19	Speech Synthesis Vocalization	Browser vocalizes recognized text through audio output.	`window.speechSynthesis` speaks utterance clearly.	**PASS**	Audio output verified
TC-20	Symmetric NAT Network Traversal	P2P connection succeeds across strict corporate firewalls.	STUN alone may fail without TURN relay in strict NAT.	**PARTIAL**	STUN verified; TURN pending
TC-21	Continuous Dynamic Sentence Translation	Model translates full grammatical ISL sentence with motion.	Model currently classifies isolated static alphabet signs only.	**NOT TESTED**	Future Phase 6 enhancement
TC-22	Gemini Generative Sentence Rewriting	Gemini restructuring partial words into formal sentences.	Proxy adapter interface defined; pending live server key.	**NOT TESTED**	Phase 5 planned feature

15.2 Confirmed Machine Learning Model Metrics

The reported test evaluation metrics on the 4,268 unseen benchmark images are:

- **Accuracy:** 97.00%
- **Loss:** 0.1045
- **Macro Precision:** 0.9739
- **Macro Recall:** 0.9703
- **Macro F1-Score:** 0.9704
- **Maximum Observed Probability Error (Keras vs. TFLite):** 2.38×10^{-7}

Chapter 16: Security, Privacy, Ethics & Accessibility

16.1 Security Architecture & Transit Encryption

SignMate implements a multi-layered security model protecting user identity, device hardware, and media streams:

SECURITY LAYER	MECHANISM & PROTOCOL	THREAT MITIGATED
Hardware Permissions	Browser Permission Sandbox (`getUserMedia`)	Prevents unauthorized eavesdropping or background camera access.
Web Delivery	HTTPS with TLS 1.3 Encryption	Eliminates Man-in-the-Middle (MitM) attacks and code tampering.
Media Plane Transport	DTLS-SRTP (Datagram TLS + Secure RTP)	Guarantees that video, audio, and sign subtitle streams cannot be intercepted on intermediate Wi-Fi routers.
Control Plane Signaling	WSS (Secure WebSockets over SSL) to Google RTDB	Prevents eavesdropping on SDP offers/answers and session tokens.
User Authentication	Firebase Auth with bcrypt-hashed credentials & JWTs	Protects against credential harvesting and session hijacking.
Backend Validation	Pydantic schemas & strict regex validation	Prevents buffer overflows, malformed base64 injection, and DoS attacks.

16.2 Privacy Policy: Ephemeral In-Memory Image Processing

> **ZERO PERSISTENCE IMAGE POLICY**
>
> A major privacy concern in video and AI platforms is whether user camera images are retained on remote servers. SignMate operates on a strict **Zero Data Retention (ZDR)** architecture: incoming video frames are converted into in-memory byte buffers in RAM, passed through the model forward pass, and immediately discarded by the Python garbage collector. **Zero user photos, video clips, or biometric data are ever written to hard drives or cloud databases.**

16.3 Ethical AI Guidelines & Avoiding Exaggerated Claims

> **ACADEMIC INTEGRITY IN AI PRESENTATIONS**
>
> **1. Ethical Representation of Sign Language:** It is unethical and academically dishonest to describe an alphabet classifier as a "complete Indian Sign Language translator". Natural ISL relies heavily on facial expressions (grammatical non-manual markers), two-handed compound gestures, and 3D spatial references. SignMate honestly positions itself as an *assistive fingerspelling recognition tool* that bridges communication gaps.
>
> **2. Skin Tone Fairness via Grayscale Normalization:** Many computer vision datasets over-represent fair skin tones. By converting incoming webcam crops into single-channel luminance grayscale, SignMate strips away racial and skin-tone color variations, focusing solely on geometric hand contours and finger edges.

16.4 Accessibility Standards (WCAG 2.1 AA Compliance)

Because SignMate is engineered for the Deaf and Hard-of-Hearing community, accessibility is integrated directly into the component system:

- **Screen Reader Live Regions (`src/utils/a11y.ts`):** Dynamic events (e.g., *"Letter H recognized"*, *"Sign recognition enabled"*, *"Partner joined"*) are automatically announced to screen readers using ARIA live regions without interrupting navigation focus.
- **High-Contrast Non-Glare Dark Theme:** High contrast palette (`#121824` background with crisp white typography and vivid cyan/blue accents) minimizes eye fatigue during prolonged video calls.
- **Keyboard Accessibility:** All buttons, modals, and sliders are fully operable using standard `Tab` , `Enter` , `Space` , and `Escape` key commands with visible focus outlines.

Chapter 17: Production Deployment Configuration & Build Process

17.1 Production Deployment Topology

SignMate utilizes a decoupled production deployment strategy, matching specific workloads to optimized cloud infrastructure:

SUBSYSTEM	HOSTING PLATFORM	RUNTIME ARTIFACT	PUBLIC ADDRESS / PORT
Web Frontend Client	Vercel Global Edge Network	Static HTML5, CSS, ES6 JavaScript bundle (`dist/`)	`https://signmate-ashy.vercel.app/`
Signaling Bus	Google Firebase RTDB	Managed WebSocket Cloud Bus	`https://signmate-a73b3-default-rtdb.firebaseio.com/`
User & Room Database	Google Cloud Firestore	Managed NoSQL Cloud Database	Direct Google SDK connection (TLS 443)
AI Inference Server	Workstation / Cloud VPS (Python)	Python 3.12/3.13 ASGI Server (`uvicorn`)	`http://localhost:8000` or Cloud Container

17.2 Vercel Continuous Deployment Configuration

Vercel is linked directly to the project's Git repository. Every code commit triggers an automated CI/CD pipeline:

- **Framework Preset:** Vite (React + TypeScript)
- **Build Command:** `npm run build` (executes `vite build`, running Rollup tree-shaking and minification)
- **Output Directory:** `dist`
- **Node Version:** 20.x / 22.x LTS
- **SSL / TLS:** Automatic wildcard certificate provisioning via Let's Encrypt with forced HTTPS redirection.

17.3 Production Environment Variables

Sensitive environment variables are configured in the Vercel dashboard and local `.env` files:

```
# Firebase Cloud Services Configuration
VITE_FIREBASE_API_KEY=AIzaSyDRvd-Br6GjHDNzVy02FrwCS0VYX8YHv_c
VITE_FIREBASE_AUTH_DOMAIN=signmate-a73b3.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=signmate-a73b3
VITE_FIREBASE_STORAGE_BUCKET=signmate-a73b3.firebasestorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=220476879234
VITE_FIREBASE_APP_ID=1:220476879234:web:c6751a68537f6b34b25f9b
VITE_FIREBASE_DATABASE_URL=https://signmate-a73b3-default-rtdb.firebaseio.com

# AI Inference Backend URL (Optional Override)
VITE_AI_API_BASE_URL=http://localhost:8000
```

17.4 Step-by-Step Local Reproduction Guide

To run and test the complete SignMate stack on a fresh development machine:

```
Terminal: PowerShell / Bash
```

```
# 1. Clone the project repository
git clone https://github.com/rashhiye/signmate-ai-for-inclusive-communication.git
cd signmate-ai-for-inclusive-communication

# 2. Install Node.js frontend dependencies
npm install

# 3. Start the Vite local development server
npm run dev
# → Local web app runs at: http://localhost:5173

# 4. In a separate terminal, install Python ML dependencies
pip install fastapi uvicorn tensorflow keras pillow numpy

# 5. Start the FastAPI Deep Learning Inference Microservice
python fastapi_server.py
# → Model server loads SignMate_stage1_best.keras and listens on: http://localhost:8000

# 6. Verify production build compilation
npm run build
npm run preview
# → Tests the exact production bundle locally on: http://localhost:4173
```

Chapter 18: Project Evolution, Design Decisions & Late Stack Migrations

18.1 Chronological Evolution of SignMate

The final architecture of SignMate is the result of iterative engineering refinements. Documenting these transitions demonstrates rigorous engineering problem-solving and maturity:

DEVELOPMENT PHASE	INITIAL PLANNED TECHNOLOGY	FINAL ADOPTED TECHNOLOGY & RATIONALE
Phase 1: Client Framework	Flutter (Dart) Mobile Application	**Migrated to React 19 + TypeScript Web App:** Eliminated native APK installation friction; achieved 100% universal access across all operating systems via standard web URLs.
Phase 2: Video Conferencing	ZEGOCLOUD Commercial SDK	**Migrated to Native WebRTC Full Mesh:** Eliminated commercial licensing costs, vendor lock-in, and proprietary SDK black boxes. Direct P2P ownership.
Phase 3: Recognition Pipeline	MediaPipe 21 Landmarks + GRU RNN	**Migrated to MobileNetV3Small CNN:** Eliminated severe landmark cascade failures; CNN processes raw pixel contours and textures directly with 97% test accuracy.
Phase 4: ML Deployment	Heavy Keras H5 Checkpoint (50MB+)	**Converted to MobileNetV3Small TFLite (3.88 MB):** Stripped training overhead; reduced artifact size for fast edge execution.
Phase 5: Signaling Bus	Local WebSocket Node Server	**Migrated to Firebase Realtime Database:** High-reliability cloud-hosted signaling bus with automatic `onDisconnect` cleanup.

18.2 Examiner Defense: "Why Did You Change Your Technology Stack?"

Examiner Question: "Why did you migrate from Flutter and ZEGOCLOUD to React and native WebRTC late in development?"

Model Candidate Answer:
"Respected examiners, during early prototyping, we evaluated Flutter and third-party video SDKs like ZEGOCLOUD. However, as our system requirements matured, we encountered three fundamental engineering roadblocks:

1. **Accessibility Barriers:** Forcing a Deaf user or a hearing doctor to download an external APK and configure permissions violated our core goal of *inclusive accessibility*. A web application opens instantly on any device via a simple URL link.
2. **Financial & Architectural Independence:** Commercial SDKs like ZEGOCLOUD impose strict usage quotas and recurring per-minute charges. By implementing native **WebRTC**, we built a fully autonomous, zero-cost, open-source peer-to-peer system.
3. **Deep Learning Reliability:** Our earlier GRU model suffered from cascading failures whenever MediaPipe lost a landmark point in dim lighting. Replacing it with **MobileNetV3Small** gave us direct visual feature extraction, jumping from fragile predictions to **97% independent test accuracy**.

Rather than clinging to flawed early assumptions, we made the disciplined engineering decision to refactor the stack into a production-grade, demonstrable web application."

18.3 Examiner Defense: "What Would You Improve If You Had Another Month?"

Examiner Question: "What would you improve if you had another month to work on SignMate?"

Model Candidate Answer:
"If granted another month of development, I would prioritize four specific enhancements:

1. **Dedicated TURN Server Infrastructure:** Deploy open-source `coturn` servers on cloud instances to guarantee 100% WebRTC connection traversal across restrictive corporate and university symmetric firewalls.
2. **Continuous Sentence Recognition:** Transition from static fingerspelling to a spatio-temporal model (such as a 3D CNN or Video Vision Transformer) capable of interpreting dynamic, two-handed continuous ISL signs with motion over time.
3. **Secure Gemini Proxy Backend:** Finalize the cloud proxy for our `GeminiService` to provide generative sentence completion and grammar restructuring without exposing API keys in the client browser.
4. **Dataset Expansion:** Collect real-world sign video samples from a diverse demographic of signers under varying real-world room lighting conditions to further boost live camera robustness."

Chapter 19: Known Limitations & Future Roadmap

19.1 Honest Technical Audit of Current Limitations

An exceptional project presentation acknowledges architectural boundaries transparently:

AREA	CURRENT REALITY / LIMITATION	UNDERLYING TECHNICAL CAUSE
Scope of Sign Vocabulary	Static Alphabet Fingerspelling (A–Z + Neutral)	Trained on single-frame static images. Continuous natural signs require multi-frame temporal modeling (3D convolutions or Transformers).
Signer Hand Dominance	Optimized primarily for right-handed signers	Dataset samples predominantly feature right-hand gestures; left-handed mirroring requires dedicated augmentation or flip training.
Network NAT Traversal	Reliant on public STUN servers; lacks dedicated TURN	Direct P2P UDP packets may be dropped behind restrictive enterprise or university symmetric NATs without a paid TURN relay.
Participant Scaling	Hard capped at **4 participants** per room	Full mesh topology scales at $O(N^2)$ connections. Exceeding 4 participants causes CPU and upload bandwidth saturation on client devices.
Lighting & Motion Sensitivity	Degrades under extreme low-light or rapid hand motion	Consumer webcams introduce motion blur and sensor noise in dim conditions, altering boundary contours expected by MobileNet.

19.2 Future Architectural Roadmap

1. **Continuous ISL Sentence Translation:** Replace single-frame image classification with a Video Vision Transformer (ViViT) or ConvLSTM model capable of processing continuous 30-frame video sequences, capturing movement trajectory, two-handed coordination, and facial grammar.
2. **SFU (Selective Forwarding Unit) Architecture:** Upgrade the WebRTC media layer from a full mesh to an SFU (e.g., MediaSoup, Pion, or LiveKit). An SFU allows each participant to upload only 1 video stream to a lightweight cloud router that duplicates and distributes packets, scaling rooms to 20+ participants effortlessly.
3. **Dedicated Coturn Infrastructure:** Provision containerized `coturn` relay servers on Google Cloud or AWS to guarantee 100% call establishment even behind strict corporate firewalls.
4. **Multilingual Speech & Caption Synthesis:** Expand voice output beyond English to regional Indian languages (Malayalam, Hindi, Tamil, Telugu) by piping recognized text through Google Cloud Translation and Indic-TTS engines.
5. **On-Device WebAssembly / WebGPU Acceleration:** Package the 3.88 MB TensorFlow Lite model directly into browser WebAssembly/WebGPU via TensorFlow.js or ONNX Runtime Web, achieving 100% client-side zero-backend deep learning execution.

Chapter 20: Complete Viva Question Bank (110 Examiner Questions & Deep Model Answers)

This chapter contains 110 meticulously prepared questions grouped into categories A through T. Every question provides a simple, direct answer, a deeper technical analysis, and warnings about potential examiner traps.

Q1 **[Cat A] What is SignMate in simple words?**

Answer: SignMate is an accessible web application that lets Deaf and hearing people have a video call where sign language hand gestures are automatically translated into text subtitles and spoken out loud.

Deeper Technical Explanation: It integrates real-time WebRTC video conferencing with a convolutional neural network (MobileNetV3Small) trained to recognize Indian Sign Language fingerspelling.

SignMate Project Example: When a user signs the letter 'A' into the camera, 'A' appears on the screen, full words like 'ALL' are suggested, and the text is vocalized for hearing participants.

Q2 **[Cat A] What exact problem does SignMate solve?**

Answer: It solves the severe communication gap between Deaf signers and hearing people during video calls, eliminating the need for expensive human interpreters or slow text typing.

Deeper Technical Explanation: Mainstream video platforms (Zoom, Google Meet) rely on voice activity detection to highlight speakers and lack any native computer vision sign interpretation tools.

SignMate Project Example: In a standard video call, a Deaf person must look down to type in a chat box. In SignMate, they simply sign naturally to the webcam.

Q3 **[Cat A] What is the main contribution of your project?**

Answer: Our main contribution is embedding deep learning sign classification and client-side dictionary word acceleration directly into an open-source, serverless WebRTC video conferencing web application.

Deeper Technical Explanation: We proved that high-accuracy (97%) sign recognition can run in real-time over the web without requiring proprietary commercial SDKs or specialized sensory hardware.

SignMate Project Example: We achieved sub-50ms inference latency and 4-participant full mesh video calling using standard web technologies.

Q4 **[Cat A] Is SignMate translating complete sign language sentences or only individual letters?**

Answer: SignMate currently classifies isolated static alphabet signs (ISL fingerspelling A–Z) plus a neutral background state, assisted by word-prediction dictionaries.

Deeper Technical Explanation: Continuous Indian Sign Language translation requires complex multi-frame temporal modeling of dynamic two-handed movements and facial expressions, which is planned for future work.

SignMate Project Example: A user fingerspells 'H-E-L' and selects 'HELLO' from the dictionary suggestion chips.

⚠️ **Examiner Trap / Status Alert:** NEVER claim SignMate translates full continuous ISL grammar. Always specify it is isolated alphabet recognition with dictionary assistance.

Q5 **[Cat A] Who are the target beneficiaries of this system?**

Answer: Deaf and Hard-of-Hearing individuals, their hearing family members, teachers in inclusive classrooms, telemedicine doctors, and customer support representatives.

Deeper Technical Explanation: It enables direct independent communication without waiting hours or paying fees for professional certified sign language interpreters.

SignMate Project Example: A Deaf student participating in a group study project with three hearing classmates.

Q6 **[Cat B] What software development life cycle (SDLC) model did you follow?**

Answer: We followed the Agile iterative prototyping model.

Deeper Technical Explanation: Agile allowed us to iterate through multiple prototypes, discover critical real-world constraints (like the unreliability of landmark-only GRUs), and refactor the architecture quickly.

SignMate Project Example: Migrating from Flutter to React and from ZEGOCLOUD to WebRTC was accomplished across discrete sprint iterations.

Q7 **[Cat B] What are the primary functional requirements of SignMate?**

Answer: User authentication, 6-character room creation, 4-participant WebRTC video calling, camera/mic controls, 120ms sign detection, temporal smoothing, dictionary suggestions, and speech synthesis.

Deeper Technical Explanation: Each requirement is formally tracked in Chapter 5 with verified source code implementations.

SignMate Project Example: FR-08 enforces that a hand sign must be held stable for 2 frames before committing to prevent jitter.

Q8 **[Cat B] What are the key non-functional requirements?**

Answer: Low latency (<50ms inference, <250ms video delay), high reliability (client-side fallback if backend drops), strong security (DTLS-SRTP encryption), and WCAG 2.1 AA accessibility.

Deeper Technical Explanation: Non-functional requirements dictate architectural choices, such as using UDP over TCP and converting heavy Keras models to TFLite.

SignMate Project Example: If the Python FastAPI server goes down, the web client seamlessly falls back to MediaPipe landmarks so the user never sees a crash.

Q9 **[Cat B] How do you ensure accessibility for DHH users?**

Answer: Through visual subtitle streaming, live landmark skeleton overlays, high-contrast dark mode, and screen-reader announcements using ARIA live regions.

Deeper Technical Explanation: WCAG 2.1 AA guidelines were implemented across semantic HTML elements and keyboard navigation handlers in src/utils/a11y.ts.

SignMate Project Example: When an AI letter is detected, announceToScreenReader('Letter H recognized') notifies blind or low-vision callers.

Q10 **[Cat C] Can you explain the high-level architecture of SignMate?**

Answer: SignMate uses a multi-tier architecture: a React 19 single-page frontend on Vercel, a Firebase Realtime Database signaling bus, a peer-to-peer WebRTC media mesh, and a Python FastAPI deep learning backend.

Deeper Technical Explanation: The control plane (signaling) is separated from the media plane (P2P video). Deep learning inference is decoupled via REST endpoints with a client-side fallback.

SignMate Project Example: Figure 3.1 in Chapter 3 shows how media streams bypass the web server and flow directly between peer browsers.

Q11 **[Cat C] Which components communicate directly and which go through a backend?**

Answer: Video and audio stream directly browser-to-browser via WebRTC. Signaling metadata passes through Firebase RTDB. AI frames are sent to FastAPI via HTTP POST.

Deeper Technical Explanation: Direct P2P communication minimizes server bandwidth costs and guarantees the lowest possible latency for high-definition video.

SignMate Project Example: When User A signs to User B, video bytes travel directly across the Internet over UDP without touching Vercel or Firebase.

Q12 **[Cat C] What is the difference between the control plane and media plane in WebRTC?**

Answer: The control plane handles call setup and signaling (who is calling, SDP codecs, IP candidates). The media plane transports the actual encrypted video and audio packets.

Deeper Technical Explanation: The control plane uses TCP/WebSockets over Firebase. The media plane uses UDP with DTLS-SRTP encryption directly between browsers.

SignMate Project Example: If Firebase disconnects after the call starts, the video stream often continues playing because the media plane is already connected.

Q13 **[Cat C] Why did you choose a client-server hybrid model for AI inference?**

Answer: To achieve maximum flexibility: heavy, high-accuracy deep learning runs on the Python FastAPI server when available, while the browser runs MediaPipe landmark rules locally if offline.

Deeper Technical Explanation: This dual-mode architecture guarantees zero downtime for users while preserving 97% model accuracy when connected to the backend.

SignMate Project Example: In src/ai/handSignDetector.ts, checkFastApiHealth() tests the server and dynamically routes frames.

Q14 **[Cat D] Why did you choose React 19 for the frontend?**

Answer: React 19 provides modern functional components, hooks, high rendering performance via the Virtual DOM, and excellent TypeScript support.

Deeper Technical Explanation: React's declarative component architecture makes managing complex real-time states (video tracks, peer arrays, recognized letters) predictable.

SignMate Project Example: RoomPage.tsx decomposes cleanly into VideoGrid, ParticipantTile, CallControls, and HandLandmarkOverlay.

Q15 **[Cat D] What role does Vite play in this project?**

Answer: Vite is the build tool and local development server. It uses native ES modules for instant server start and Rollup for optimized production bundling.

Deeper Technical Explanation: Unlike older Webpack configurations that took 30+ seconds to bundle, Vite provides near-instant Hot Module Replacement (HMR).

SignMate Project Example: Running 'npm run build' outputs tree-shaken, minified JavaScript files in the dist/ folder for Vercel deployment.

Q16 **[Cat D] How is state managed across the website?**

Answer: We used custom React Context hooks: useAuth for user sessions, useMediaDevices for camera/mic, useSignAI for AI buffers, and useToast for alerts.

Deeper Technical Explanation: Lightweight Context hooks eliminate the need for bulky external libraries like Redux while keeping state modular and easy to debug.

SignMate Project Example: When useSignAI commits a letter, RoomPage and RecognitionPanel re-render instantly without passing props down 10 levels.

Q17 **[Cat D] What is the difference between videoFit 'cover' and 'contain' in RoomPage?**

Answer: 'Cover' scales the video to fill the tile completely (may crop edges). 'Contain' letterboxes the video so 100% of the camera frame is visible.

Deeper Technical Explanation: Contain mode is critical for signers because cover mode might crop their hands or arms out of view on narrow mobile screens.

SignMate Project Example: Users can toggle between cover and contain modes via the call settings menu in RoomPage.tsx.

Q18 **[Cat E] What does WebRTC stand for and what is its main advantage?**

Answer: Web Real-Time Communication. Its main advantage is browser-native peer-to-peer audio and video streaming with sub-200ms latency without plugins.

Deeper Technical Explanation: It provides standard C++ networking implementations built directly into Chromium and WebKit browsers.

SignMate Project Example: SignMate uses RTCPeerConnection to stream 720p webcam video between callers.

Q19 **[Cat E] What is an SDP Offer and SDP Answer?**

Answer: SDP (Session Description Protocol) is a text format describing media capabilities. The Offer lists codecs and encryption parameters from Caller A; the Answer is Caller B's response.

Deeper Technical Explanation: SDP exchange negotiates video codecs (e.g., VP8/H.264), audio codecs (Opus), resolution, and SRTP cryptographic fingerprints.

SignMate Project Example: In WebRTCService.ts, peer.pc.createOffer() produces the SDP string sent to Firebase RTDB.

Q20 **[Cat E] What are ICE candidates?**

Answer: ICE (Interactive Connectivity Establishment) candidates are network endpoints (IP address, port, protocol) where a browser can potentially be reached.

Deeper Technical Explanation: The browser discovers Host candidates (local LAN IP), Server Reflexive candidates (public IP from STUN), and Relay candidates (from TURN).

SignMate Project Example: peer.pc.onicecandidate captures candidates and stores them in Firebase RTDB so the remote peer can test connectivity.

Q21 **[Cat F] What is the purpose of a STUN server?**

Answer: STUN (Session Traversal Utilities for NAT) tells a browser its own public IP address and port as seen from the public Internet when behind a NAT router.

Deeper Technical Explanation: Home Wi-Fi routers use NAT, giving laptops private IPs (like 192.168.1.5). STUN allows the laptop to find out its public Internet IP.

SignMate Project Example: SignMate uses Google's public STUN server pool: stun:stun.l.google.com:19302.

Q22 **[Cat F] When is a TURN server necessary?**

Answer: A TURN (Traversal Using Relays around NAT) server is needed when both peers are behind strict 'Symmetric NATs' or corporate firewalls that block direct P2P UDP packets.

Deeper Technical Explanation: TURN relays all audio/video packets through a cloud server. It is expensive to run because it consumes gigabytes of server bandwidth.

SignMate Project Example: In SignMate, public STUN is implemented and verified; TURN is planned for restrictive enterprise network deployments.

Q23 **[Cat F] Why is signaling needed if WebRTC is peer-to-peer?**

Answer: Because two browsers on opposite sides of the world do not know each other's IP addresses, security keys, or codecs before the call starts.

Deeper Technical Explanation: WebRTC deliberately omits signaling from the specification, allowing developers to choose any signaling protocol (WebSockets, Firebase, SIP).

SignMate Project Example: SignMate uses Firebase Realtime Database as its cloud signaling server to pass the initial SDP offer and answer.

Q24 **[Cat F] How did you solve 'glare' in WebRTC signaling?**

Answer: Glare happens when two peers send each other an SDP offer at the exact same moment. We resolved this deterministically by comparing client IDs.

Deeper Technical Explanation: The client with the lexicographically smaller ID (clientId_A < clientId_B) acts as the initiator and creates the offer; the other client answers.

SignMate Project Example: In WebRTCService.ts: const isInitiator = this.clientId < remoteId; completely eliminates glare.

Q25 **[Cat F] How many peer connections are required for a 4-person mesh call?**

Answer: Exactly 6 bidirectional peer connections across the room, with each individual participant maintaining 3 peer connections.

Deeper Technical Explanation: The mathematical formula for a full mesh is N * (N - 1) / 2. For N = 4: (4 * 3) / 2 = 6 total connections.

SignMate Project Example: Each participant uploads 3 video streams and downloads 3 video streams simultaneously.

Q26 **[Cat G] What is Machine Learning in simple terms?**

Answer: Machine Learning is a method where a computer program learns patterns from data examples rather than relying on human programmers to write explicit rules.

Deeper Technical Explanation: The model adjusts numerical weights through mathematical optimization (gradient descent) to minimize prediction error.

SignMate Project Example: We showed the model 12,845 photos of hand signs so it learned what letter 'A' looks like.

Q27 **[Cat G] What is an Artificial Neural Network (ANN)?**

Answer: A computational model inspired by biological brains, consisting of interconnected layers of artificial neurons that process and transform input numbers.

Deeper Technical Explanation: Each neuron multiplies inputs by weights, adds a bias, and passes the result through an activation function (like ReLU or Softmax).

SignMate Project Example: SignMate uses a Deep Convolutional Neural Network with over 1.17 million parameters.

Q28 **[Cat H] What is a Convolutional Neural Network (CNN)?**

Answer: A specialized neural network designed for processing visual images using small sliding filters to preserve 2D spatial relationships.

Deeper Technical Explanation: Unlike standard dense networks that flatten images into 1D arrays, CNNs detect localized features like edges, curves, and textures regardless of position.

SignMate Project Example: In SignMate, convolutional layers detect finger joints, thumb curves, and palm contours from the 160×160 image.

Q29 **[Cat H] Why did you choose MobileNetV3Small over ResNet or VGG?**

Answer: MobileNetV3Small was specifically designed by Google for real-time mobile and edge vision. It has only ~1.17M parameters compared to ResNet-50's 25M+ and VGG-16's 138M.

Deeper Technical Explanation: It uses depthwise separable convolutions and squeeze-and-excitation optimization to achieve high accuracy with minimal CPU and memory usage.

SignMate Project Example: MobileNetV3Small executes inference in ~34ms on standard CPUs, allowing real-time video processing.

Q30 **[Cat H] What is Transfer Learning?**

Answer: Transfer learning is taking a model that was already trained on a massive general dataset (ImageNet) and fine-tuning it for our specific task (sign language).

Deeper Technical Explanation: Instead of training 1.17M weights from scratch (which requires months and millions of images), the model reuses pre-learned edge and shape features.

SignMate Project Example: We used ImageNet pretrained weights for MobileNetV3Small and trained only the top classification head.

Q31 **[Cat H] Why did you freeze the base model during initial training?**

Answer: Freezing the base model prevents ImageNet's valuable pre-trained feature weights from being corrupted or destroyed by large random gradients during early epochs.

Deeper Technical Explanation: Only the 77,339 weights in our custom classification head were updated, allowing stable, fast convergence in just 7 epochs.

SignMate Project Example: Non-trainable params: 939,120; Trainable params: 77,339.

Q32 **[Cat H] What is the function of GlobalAveragePooling2D?**

Answer: It compresses spatial feature maps (5×5×576) into a 1D vector (576 elements) by calculating the mathematical average of each feature map.

Deeper Technical Explanation: It drastically reduces parameter count compared to Flatten layers, prevents overfitting, and makes the model invariant to slight spatial translations.

SignMate Project Example: Output shape changes from (None, 5, 5, 576) to (None, 576).

Q33 **[Cat H] What does the Dropout layer do?**

Answer: Dropout randomly turns off a percentage (20%) of neurons during training to prevent the network from becoming overly reliant on any single feature.

Deeper Technical Explanation: It forces the network to learn redundant, robust feature combinations, acting as an effective regularization technique against overfitting.

SignMate Project Example: Our model uses Dropout(0.2) before Dense(128) and before the output layer.

Q34 **[Cat H] What does the Softmax activation function do?**

Answer: Softmax converts the model's raw numerical outputs (logits) into a probability distribution where each value is between 0 and 1, and all 27 values sum to 1.0.

Deeper Technical Explanation: The formula is: exp(z_i) / sum(exp(z_j)). The index with the highest probability represents the model's final predicted class.

SignMate Project Example: For letter 'A', the model outputs a vector where index 1 has value 0.98, and other classes have near-zero values.

Q35 **[Cat I] Describe your training and testing dataset.**

Answer: The dataset contains 17,113 total images across 27 classes: 12,845 images in the training set and 4,268 images in the independent testing set.

Deeper Technical Explanation: Images are single-channel grayscale JPEGs with raw dimensions around 310×310, resized to 160×160 pixels.

SignMate Project Example: Class 0 is background/no-hand; Classes 1 to 26 represent letters A to Z.

Q36 **[Cat I] Why is the background/no-hand class (Class 0) so important?**

Answer: Without a background class, a 26-class model is mathematically forced to predict a letter even when the camera sees an empty room or a resting hand.

Deeper Technical Explanation: Class 0 provides a negative control, allowing the model to assign high probability to 'neutral' when no valid sign gesture is made.

SignMate Project Example: In signmate_labels.json, '0' maps to '' (empty string), so background frames are silently ignored.

Q37 **[Cat I] Why did you use grayscale instead of RGB color images?**

Answer: Grayscale reduces memory usage by 66.7% (1 channel instead of 3) and prevents the model from developing bias based on human skin color tones.

Deeper Technical Explanation: Hand sign recognition depends on geometric contours and finger positions, which are fully captured in luminance grayscale.

SignMate Project Example: Input tensor shape is [1, 160, 160, 1].

Q38 **[Cat I] How does the model handle grayscale input if MobileNet expects 3 channels?**

Answer: We included a Concatenate layer (concatenate_1) inside the model graph that duplicates the single grayscale channel three times before feeding MobileNet.

Deeper Technical Explanation: This allows the pre-trained ImageNet weights (which expect 3-channel input) to operate without modifying the internal convolutional filter shapes.

SignMate Project Example: In model summary: Input (None, 160, 160, 1) → Concatenate (None, 160, 160, 3) → MobileNetV3Small.

Q39 **[Cat I] What data augmentations were applied during training?**

Answer: RandomRotation(0.05), RandomTranslation(0.05, 0.05), RandomZoom(0.08), and RandomContrast(0.08).

Deeper Technical Explanation: Augmentations artificially simulate variations in user camera distance, slight hand tilts, and lighting differences, preventing overfitting.

SignMate Project Example: These operations run directly inside the Keras Sequential augmentation layer during training.

Q40 **[Cat I] What was the double-normalization bug and how was it fixed?**

Answer: The model was built with include_preprocessing=True, which already rescales pixel values internally. Dividing by 255 in external code caused double normalization.

Deeper Technical Explanation: Dividing by 255 twice reduced pixel values to near zero (0.0 to 0.0039), destroying feature signals and causing erratic predictions.

SignMate Project Example: The fix was ensuring fastapi_server.py feeds raw unscaled float32 values in the range 0 to 255.

Q41 **[Cat J] What optimizer and loss function were used?**

Answer: Adam optimizer (Adaptive Moment Estimation) and Categorical Cross-Entropy loss.

Deeper Technical Explanation: Adam computes adaptive learning rates for each parameter using moving averages of past gradients and squared gradients.

SignMate Project Example: Categorical cross-entropy penalizes the model based on the negative log probability assigned to the true class.

Q42 **[Cat J] What is an epoch and how many epochs did training run?**

Answer: An epoch is one complete training pass through all 12,845 training images. Training stopped at Epoch 7 due to early stopping.

Deeper Technical Explanation: The best checkpoint was saved at Epoch 4 where validation loss reached 0.0078 and validation accuracy reached 100%.

SignMate Project Example: The model was saved as SignMate_stage1_best.keras.

Q43 **[Cat J] What is Early Stopping and why did you use it?**

Answer: Early Stopping is a callback that monitors validation loss and stops training if it fails to improve for a set number of epochs (patience).

Deeper Technical Explanation: It prevents overfitting by halting training before the model starts memorizing noise in the training set.

SignMate Project Example: Training halted at Epoch 7 because validation loss had stabilized at Epoch 4.

Q44 **[Cat J] What is the difference between an epoch, a batch, and an iteration?**

Answer: An epoch is one pass through the entire dataset. A batch is the subset of images evaluated together (e.g., 32). An iteration is one weight update per batch.

Deeper Technical Explanation: If a dataset has 12,845 images and batch size is 32, one epoch contains approximately 402 iterations (weight updates).

SignMate Project Example: Total training iterations across 7 epochs: ~2,800 gradient updates.

Q45 **[Cat J] What is a learning rate?**

Answer: The learning rate is a hyperparameter that controls how large of an adjustment is made to the model's weights during gradient descent.

Deeper Technical Explanation: Too high a learning rate causes training to oscillate or diverge; too low a learning rate makes training excessively slow or gets stuck in local minima.

SignMate Project Example: A learning rate of 0.001 was used with the Adam optimizer.

Q46 **[Cat J] What is Overfitting and how did you prevent it?**

Answer: Overfitting is when a model performs exceptionally well on training data but poorly on unseen test data because it memorized specific training examples.

Deeper Technical Explanation: We prevented overfitting using: 1) Data augmentation, 2) Dropout layers (0.2), 3) Global Average Pooling, 4) Early Stopping callbacks, and 5) Transfer learning.

SignMate Project Example: Proof of no catastrophic overfitting: The model achieved 97.00% accuracy on 4,268 completely unseen test images.

Q47 **[Cat J] What is Underfitting?**

Answer: Underfitting occurs when a model is too simple to learn the underlying patterns in the data, resulting in poor performance on both training and test data.

Deeper Technical Explanation: It is caused by insufficient model capacity, overly aggressive regularization, or premature training termination.

SignMate Project Example: Our high training and validation accuracy confirmed the model had ample capacity to learn the 27 classes.

Q48 **[Cat J] What is the difference between parameters and hyperparameters?**

Answer: Parameters are internal weights learned automatically by the model from data (e.g., the 1.17M weights). Hyperparameters are settings chosen by the engineer before training.

Deeper Technical Explanation: Hyperparameters include learning rate, batch size, number of epochs, dropout rate, and image input size.

SignMate Project Example: Hyperparameter: input size 160×160; Learned parameter: Dense weight matrices.

Q49 **[Cat J] Why did you use Categorical Cross-Entropy instead of Binary Cross-Entropy?**

Answer: Binary Cross-Entropy is used for 2-class problems. Categorical Cross-Entropy is mathematically required for multi-class classification where classes are mutually exclusive.

Deeper Technical Explanation: It compares the predicted 27-element Softmax probability vector against the one-hot encoded ground truth label vector.

SignMate Project Example: Loss = -sum(y_true * log(y_pred)).

Q50 **[Cat J] What is One-Hot Encoding?**

Answer: A representation where a categorical label is converted into a vector of zeros with a single '1' at the index corresponding to the class.

Deeper Technical Explanation: Class 0 is [1, 0, 0, ...], Class 1 ('A') is [0, 1, 0, ...], Class 2 ('B') is [0, 0, 1, ...].

SignMate Project Example: It ensures the model treats class labels as distinct categories rather than imposing an artificial numerical ordering.

Q51 **[Cat J] What was your final test accuracy and test loss?**

Answer: On the 4,268 unseen test images, the model achieved 97.00% test accuracy and a test loss of 0.1045.

Deeper Technical Explanation: Macro Precision was 0.9739, Macro Recall was 0.9703, and Macro F1-score was 0.9704.

SignMate Project Example: These are verified historical evaluation metrics recorded from test set evaluation.

Q52 **[Cat J] Why is validation accuracy 100% at epoch 4 but test accuracy 97%?**

Answer: Validation batches during training shared similar capture conditions with the training data, while the test set featured distinct signers, lighting, and camera angles.

Deeper Technical Explanation: A 3% difference between validation and completely independent test data is standard in real-world computer vision and demonstrates healthy generalization.

SignMate Project Example: The model correctly classified 4,140 out of 4,268 test images.

Q53 **[Cat K] What is TensorFlow Lite and why convert to it?**

Answer: TensorFlow Lite is an optimized runtime for edge and mobile inference. We convert to TFLite to reduce file size from 5.27 MB to 3.88 MB and enable low-latency execution.

Deeper Technical Explanation: TFLite strips training graphs, backprop operators, and optimizer momentum states, packaging only inference kernels into an efficient FlatBuffer schema.

SignMate Project Example: SignMate_MobileNetV3Small.tflite is 4,070,012 bytes (~3.88 MB).

Q54 **[Cat K] What is the difference between training and inference?**

Answer: Training is the heavy phase where the model sees labeled data, calculates errors, and updates its weights. Inference is the fast production phase where the fixed model predicts on new unseen images.

Deeper Technical Explanation: Training requires massive GPU memory and backward passes; inference requires only a forward pass without updating any weights.

SignMate Project Example: Training took ~7 epochs on a GPU; live inference takes ~34 milliseconds on a standard CPU.

Q55 **[Cat K] What is quantization in TensorFlow Lite?**

Answer: Quantization reduces the precision of model weights from 32-bit floating point (float32) down to 16-bit float (float16) or 8-bit integers (int8).

Deeper Technical Explanation: It can shrink model size by up to 75% and accelerate CPU inference with negligible loss in accuracy.

SignMate Project Example: Our model currently runs in standard float32 mode with int8 quantization planned for mobile edge execution.

Q56 **[Cat K] How does the model handle confidence thresholds?**

Answer: If the highest predicted class probability is below 0.70 (70%), the prediction is discarded as uncertain or transitional noise.

Deeper Technical Explanation: This ensures that slight hand movements between signs do not trigger random unwanted characters.

SignMate Project Example: In RoomPage.tsx: if (result.confidence >= 0.70) validates the threshold.

Q57 **[Cat K] What is temporal smoothing?**

Answer: Temporal smoothing requires a sign to remain consistent across multiple consecutive frames (here, 2 frames ≈ 240–360ms) before committing.

Deeper Technical Explanation: It acts as a low-pass filter on predictions, eliminating single-frame classification spikes or camera sensor flicker.

SignMate Project Example: If the model predicts 'A' for one frame and immediately drops it, 'A' is not appended to the text buffer.

Q58 **[Cat K] What is debouncing in the context of SignMate?**

Answer: Debouncing prevents repeatedly holding the same sign from adding duplicate letters on every single frame.

Deeper Technical Explanation: Without debouncing, holding sign 'A' for 2 seconds would produce 16 'A's. Debouncing locks the committed letter until the hand is relaxed or held for a deliberate duration.

SignMate Project Example: In RoomPage.tsx, lastCommittedLetter locks the character until released.

Q59 **[Cat K] How does a user intentionally sign a double letter like 'LL' in 'HELLO'?**

Answer: Through the hold-count mechanism: if a user sustains the exact same gesture continuously for 13 frames (~1.5 seconds), the system recognizes the deliberate hold and appends the second letter.

Deeper Technical Explanation: This balances typing speed with intentional spelling of double-letter words.

SignMate Project Example: In RoomPage.tsx: if (holdCount >= 13 && repeatCount === 0) appends the second 'L'.

Q60 **[Cat K] How does the system know when a user finished a sign?**

Answer: When the hand is lowered, opened into a flat resting palm, or leaves the camera frame for 2 consecutive frames, releaseCount triggers a reset.

Deeper Technical Explanation: This resets the candidate letter and hold counters, making the system immediately receptive to the next sign.

SignMate Project Example: In RoomPage.tsx lines 370–384.

Q61 **[Cat L] Why is accuracy alone not sufficient to evaluate a classifier?**

Answer: Accuracy can be misleading on imbalanced datasets. If 90% of data is background and 10% is signs, a dummy model predicting 'background' every time has 90% accuracy but is completely useless.

Deeper Technical Explanation: Metrics like Precision, Recall, and F1-score reveal class-by-class performance and expose bias.

SignMate Project Example: We measured Macro Precision (0.9739), Macro Recall (0.9703), and Macro F1 (0.9704) across all 27 individual classes.

Q62 **[Cat L] What is Precision in the context of SignMate?**

Answer: Out of all times the model predicted letter 'A', what percentage was truly letter 'A'? High precision means low false alarms.

Deeper Technical Explanation: Formula: TP / (TP + FP).

SignMate Project Example: SignMate achieved a Macro Precision of 97.39% on the test benchmark.

Q63 **[Cat L] What is Recall in the context of SignMate?**

Answer: Out of all actual images of letter 'A' in the test set, what percentage did the model correctly identify? High recall means few missed signs.

Deeper Technical Explanation: Formula: TP / (TP + FN).

SignMate Project Example: SignMate achieved a Macro Recall of 97.03% on the test benchmark.

Q64 **[Cat L] What is the F1-Score?**

Answer: The F1-score is the harmonic mean of precision and recall: 2 * (Precision * Recall) / (Precision + Recall).

Deeper Technical Explanation: It provides a balanced measure that penalizes extreme imbalances between precision and recall.

SignMate Project Example: SignMate's Macro F1-score is 0.9704.

Q65 **[Cat L] What is a Confusion Matrix?**

Answer: A 27×27 grid where rows represent ground-truth classes and columns represent predicted classes. Diagonal cells represent correct classifications.

Deeper Technical Explanation: Off-diagonal cells reveal specific pairs of signs that the model confuses with one another.

SignMate Project Example: It showed that letters with similar hand shapes (like 'M' and 'N') occasionally overlap.

Q66 **[Cat L] Why might letters like 'M' and 'N' be confused by the model?**

Answer: In Indian Sign Language, 'M' and 'N' differ primarily by thumb placement under three fingers vs. two fingers, which can look identical from certain webcam angles.

Deeper Technical Explanation: When viewed head-on without 3D depth, finger occlusion obscures thumb tucking.

SignMate Project Example: This is why dictionary prefix matching is invaluable: if 'M' or 'N' is slightly ambiguous, the word suggestion engine guides the user to the correct word.

Q67 **[Cat L] What is the difference between Macro F1 and Weighted F1?**

Answer: Macro F1 computes the unweighted average of F1-scores across all 27 classes, treating every class equally. Weighted F1 weights each score by the number of samples in that class.

Deeper Technical Explanation: Macro F1 is harsher because poor performance on a rare class significantly drags down the score.

SignMate Project Example: SignMate's Macro F1 is 0.9704; Weighted F1 is 0.9702.

Q68 **[Cat L] Does 97% test accuracy mean 97% of live webcam signs are recognized?**

Answer: No. Benchmark test sets are photographed under controlled conditions, whereas live webcams suffer from room lighting shifts, motion blur, and background clutter.

Deeper Technical Explanation: Real-time webcam accuracy is typically 80–90% depending on camera quality and lighting.

SignMate Project Example: We mitigate this with temporal smoothing and dictionary recommendations.

Q69 **[Cat M] What is FastAPI and why is it used?**

Answer: FastAPI is a modern, high-speed Python web framework used to serve our Keras/TensorFlow deep learning model via REST endpoints.

Deeper Technical Explanation: It provides asynchronous execution (ASGI), automatic Pydantic request validation, and sub-millisecond routing overhead.

SignMate Project Example: It runs on port 8000 and serves POST /api/v1/predict/base64.

Q70 **[Cat M] What is a REST API?**

Answer: Representational State Transfer (REST) is an architectural style for web services using standard HTTP verbs (GET, POST, PUT, DELETE) and JSON data payloads.

Deeper Technical Explanation: It decouples client and server: the React web app can run anywhere and communicate with the Python API over standard HTTP.

SignMate Project Example: React sends POST requests with Base64 image payloads and receives JSON prediction results.

Q71 **[Cat M] What is CORS and why did you configure it in FastAPI?**

Answer: Cross-Origin Resource Sharing (CORS) is a browser security mechanism that blocks web pages on one domain from making fetch requests to a different domain.

Deeper Technical Explanation: Because our frontend is on Vercel (signmate-ashy.vercel.app) while the API runs on localhost or a custom server, CORS middleware is required to allow requests.

SignMate Project Example: app.add_middleware(CORSMiddleware, allow_origins=['*']) in fastapi_server.py.

Q72 **[Cat M] What does the /api/v1/health endpoint return?**

Answer: It returns server operational status, model loaded confirmation, target input shape, and total class count.

Deeper Technical Explanation: Used by the frontend to detect whether the deep learning server is active or if it should use client-side MediaPipe rules.

SignMate Project Example: Returns {'status': 'online', 'model_loaded': true, 'input_shape': [1, 160, 160, 1], 'classes_count': 27}.

Q73 **[Cat M] What is Pydantic and how is it used in FastAPI?**

Answer: Pydantic is a Python data validation library that enforces type hints and validates incoming request schemas at runtime.

Deeper Technical Explanation: It automatically parses JSON request bodies, validates fields, and generates detailed HTTP 422 errors for malformed requests.

SignMate Project Example: class PredictBase64Request(BaseModel): image_base64: str.

Q74 **[Cat M] How is latency measured in the inference endpoint?**

Answer: Using Python's time.time() before and after image preprocessing and model forward pass, converted to milliseconds.

Deeper Technical Explanation: Latency = (time.time() - start_time) * 1000.0, returned in the JSON response as inference_time_ms.

SignMate Project Example: Typical execution latency is 30–45ms per frame on modern multi-core CPUs.

Q75 **[Cat M] What happens if an invalid or corrupt image is sent to the API?**

Answer: PIL Image.open() raises an exception, which is caught and returned as an HTTP 500/400 error without crashing the server.

Deeper Technical Explanation: The frontend catches the error and silently drops the frame without interrupting the video call.

SignMate Project Example: In fastapi_server.py: try/except block around image processing.

Q76 **[Cat N] What services do you use from Firebase?**

Answer: Firebase Authentication (user login/signup), Firebase Realtime Database (WebRTC signaling), and Cloud Firestore (room records).

Deeper Technical Explanation: It provides managed, scalable cloud backends without requiring us to deploy and maintain custom database or auth servers.

SignMate Project Example: Configured in src/firebase/config.ts.

Q77 **[Cat N] How does Firebase Authentication work in SignMate?**

Answer: Users register with email/password or Google OAuth. Firebase securely stores credentials, hashes passwords with bcrypt, and issues a JWT session token.

Deeper Technical Explanation: The token is stored in the browser's IndexedDB, keeping the user logged in across page refreshes.

SignMate Project Example: In src/firebase/firebaseAuth.ts: signInWithEmailAndPassword() and signInWithPopup().

Q78 **[Cat N] What is the difference between Authentication and Authorization?**

Answer: Authentication verifies identity (Who are you? E.g., verifying password). Authorization verifies permissions (What are you allowed to access? E.g., can this user delete this room?).

Deeper Technical Explanation: Authentication is handled by Firebase Auth; Authorization is enforced by Firestore Security Rules.

SignMate Project Example: In firestore.rules: allow delete: if request.auth.uid == resource.data.hostId ensures only the room host can delete the room.

Q79 **[Cat N] Why is Firebase Realtime Database used for WebRTC signaling?**

Answer: RTDB uses low-latency persistent WebSockets. When a client writes an SDP offer or ICE candidate, Google's cloud pushes it to other connected clients in under 30ms.

Deeper Technical Explanation: It provides the real-time event bus needed for browser discovery before WebRTC connects.

SignMate Project Example: Path: rooms/{roomId}/mesh/{pairKey}/offer.

Q80 **[Cat N] What is onDisconnect() in Firebase RTDB?**

Answer: It is a server-side hook that automatically executes a database write (such as deleting a user's presence node) the instant their TCP socket disconnects.

Deeper Technical Explanation: It prevents ghost participants from lingering in rooms if a user abruptly closes their browser tab or loses power.

SignMate Project Example: onDisconnect(myPeerRef).remove() in WebRTCService.ts.

Q81 **[Cat N] Are your Firebase API keys safe in the client source code?**

Answer: Yes. Firebase client API keys are public routing identifiers, not administrative secrets. Real security is enforced by Firestore Security Rules and Auth JWT tokens on Google's servers.

Deeper Technical Explanation: Sensitive secrets like the Firebase Admin SDK private key are never exposed in the web client.

SignMate Project Example: Firebase documentation explicitly confirms that client config objects are designed to be public.

Q82 **[Cat O] What is the role of the dictionary service in SignMate?**

Answer: It provides high-speed prefix search across 95,000+ words to recommend full conversational words from partial fingerspelled letters.

Deeper Technical Explanation: It reduces the physical effort required by Deaf signers by over 60%, allowing them to select words after signing only 2–3 letters.

SignMate Project Example: When 'HEL' is signed, it suggests 'HELLO' and 'HELP' in under 2 milliseconds.

Q83 **[Cat O] Why are priority conversational words ranked first?**

Answer: Common phrases like 'HELLO', 'PLEASE', 'THANK YOU', and 'DOCTOR' occur much more frequently in daily life than obscure dictionary terms like 'HELIUM' or 'HELIX'.

Deeper Technical Explanation: Prioritizing conversational words places the most relevant communicative terms at the top of the recommendation chips.

SignMate Project Example: Defined in PRIORITY_WORDS in src/services/dictionaryService.ts.

Q84 **[Cat O] Does Google Gemini recognize hand signs in SignMate?**

Answer: No. Gemini is a Large Language Model and is never used for image classification. Sign recognition is performed entirely by MobileNetV3Small.

Deeper Technical Explanation: Gemini's planned role is generative sentence auto-completion from recognized word sequences.

SignMate Project Example: Examiners frequently test whether candidates mistakenly believe an LLM is doing computer vision classification.

⚠️ **Examiner Trap / Status Alert:** Never say Gemini recognizes signs. MobileNetV3Small recognizes signs.

Q85 **[Cat O] What is the current implementation status of Gemini in SignMate?**

Answer: Gemini is architected as an abstract interface (IGeminiService in src/services/GeminiService.ts) and marked as a Phase 5 enhancement.

Deeper Technical Explanation: In production, language assistance is actively powered by our fast client-side 95,000-word dictionary engine to protect API keys from exposure.

SignMate Project Example: geminiSuggestionsService.ts currently throws a placeholder notice for Phase 5.

⚠️ **Examiner Trap / Status Alert:** Do not claim Gemini is live in production. It is an architected Phase 5 enhancement.

Q86 **[Cat O] Why should recognized raw text remain visible even when suggestions exist?**

Answer: To preserve user control and communication transparency. The signer should always see the exact letters they formed so they can correct mistakes.

Deeper Technical Explanation: Replacing raw input automatically without user confirmation can lead to frustrating auto-correct errors.

SignMate Project Example: The raw fingerspelling buffer is displayed alongside the clickable suggestion chips.

Q87 **[Cat O] How does Text-to-Speech (TTS) work in SignMate?**

Answer: Using the browser's native Web Speech API (window.speechSynthesis) to convert recognized text strings into spoken voice audio.

Deeper Technical Explanation: It creates a SpeechSynthesisUtterance object and plays it through the device speakers at rate 1.0.

SignMate Project Example: speakRecognizedText() in useSignAI.ts.

Q88 **[Cat P] Where is the website hosted and how is it deployed?**

Answer: The frontend is deployed on Vercel's global edge network at https://signmate-ashy.vercel.app/. Deployment is automated via Git CI/CD.

Deeper Technical Explanation: Every push to the main Git branch triggers Vite compilation, static asset generation, and global CDN caching.

SignMate Project Example: Verified live production URL: https://signmate-ashy.vercel.app/.

Q89 **[Cat P] What does Vercel host and what does it NOT host?**

Answer: Vercel hosts the static compiled web bundle (HTML, CSS, JS chunks, public JSON files). It does NOT host long-running Python TensorFlow processes.

Deeper Technical Explanation: Vercel is a serverless static/Node CDN platform. Machine learning inference runs either client-side or on a separate Python server.

SignMate Project Example: The Python FastAPI backend (fastapi_server.py) runs on a dedicated host machine or container.

Q90 **[Cat P] What command builds the project for production?**

Answer: 'npm run build', which executes 'vite build'.

Deeper Technical Explanation: It transpiles TypeScript, bundles React components with Rollup, minifies code, and outputs static assets into the dist/ directory.

SignMate Project Example: The dist/ folder contains index.html and minified JS/CSS chunks.

Q91 **[Cat P] What is the difference between production and localhost?**

Answer: Localhost runs on your local machine for development (with Hot Module Replacement). Production runs on a globally distributed CDN with minified bundles, HTTPS, and caching.

Deeper Technical Explanation: In production, WebRTC must navigate public NAT routers across the Internet using STUN servers.

SignMate Project Example: Localhost uses port 5173 (Vite) and port 8000 (FastAPI).

Q92 **[Cat P] Why is HTTPS strictly required for WebRTC?**

Answer: Modern web browsers enforce HTTPS for all sensitive hardware access APIs (getUserMedia for camera and microphone).

Deeper Technical Explanation: On non-secure HTTP origins (except localhost), the browser automatically blocks camera and microphone access.

SignMate Project Example: Vercel automatically provisions SSL/TLS certificates via Let's Encrypt.

Q93 **[Cat P] How do you test the application across two different devices?**

Answer: Both devices open the production URL (https://signmate-ashy.vercel.app/), navigate to the same room code (e.g., X8LN7A), and grant camera access.

Deeper Technical Explanation: Firebase RTDB handles signaling across the Internet, and WebRTC establishes direct P2P media streaming.

SignMate Project Example: Tested between a Windows laptop and an Android mobile phone.

Q94 **[Cat Q] How is video and audio data secured in transit?**

Answer: Through DTLS-SRTP (Datagram Transport Layer Security + Secure Real-time Transport Protocol), which provides mandatory end-to-end encryption for all WebRTC streams.

Deeper Technical Explanation: Intermediate routers or Wi-Fi eavesdroppers cannot intercept or decode the audio and video packets.

SignMate Project Example: Mandated by the W3C WebRTC standard.

Q95 **[Cat Q] Does SignMate store user webcam images on a server?**

Answer: No. SignMate operates on a zero-retention privacy policy. Video frames are processed in-memory in RAM and immediately discarded.

Deeper Technical Explanation: No camera frames, video clips, or biometric data are ever written to disk or saved to a database.

SignMate Project Example: Documented in Chapter 16 and SecurityPage.tsx.

Q96 **[Cat Q] What are the ethical considerations in sign language AI?**

Answer: 1) Avoiding exaggerated accuracy claims; 2) Respecting the Deaf community by not claiming fingerspelling replaces rich natural sign language; 3) Ensuring algorithmic fairness across diverse skin tones.

Deeper Technical Explanation: We used grayscale images to eliminate racial and skin-tone bias in the computer vision pipeline.

SignMate Project Example: We transparently disclose that the model classifies static alphabet signs, not complete sign language grammar.

Q97 **[Cat Q] How do you handle camera and microphone consent?**

Answer: The browser natively prompts the user for camera and microphone access before any stream is captured. Users can revoke permissions at any time.

Deeper Technical Explanation: If a user denies permission, the app catches the error gracefully and displays a helpful banner instead of crashing.

SignMate Project Example: useMediaDevices.ts handles permission exceptions.

Q98 **[Cat Q] How are room codes secured against brute-force guessing?**

Answer: Room codes use 6 uppercase alphanumeric characters (36^6 = ~2.17 billion combinations). Rooms expire and are cleaned up upon disconnection.

Deeper Technical Explanation: Firestore rules restrict operations to active rooms and valid participant limits.

SignMate Project Example: generateRoomCode(6) in src/utils/roomCode.ts.

Q99 **[Cat R] How did you test the WebRTC mesh connection locally?**

Answer: Using BroadcastChannel and multi-tab testing, as well as multiple devices on the same local Wi-Fi network connecting through Firebase RTDB.

Deeper Technical Explanation: BroadcastChannel allows instant tab-to-tab signaling with zero network latency, making it ideal for rapid UI and track debugging.

SignMate Project Example: Layer 1 signaling in WebRTCService.ts.

Q100 **[Cat R] How did you debug video playback issues during development?**

Answer: Using browser developer tools, chrome://webrtc-internals to inspect ICE states and packet loss, and adding catch blocks on HTMLVideoElement.play().

Deeper Technical Explanation: Browsers block auto-playing video with audio unless muted or triggered by a user gesture. We added playsInline and error handlers.

SignMate Project Example: videoRef.current.play().catch(err ⇒ console.warn(err)).

Q101 **[Cat R] How did you verify that the TFLite model matches the Keras model?**

Answer: We ran both models on identical test sample images and compared output probabilities. The maximum probability difference was ~2.38 × 10^-7.

Deeper Technical Explanation: This confirmed that the TensorFlow Lite conversion was numerically identical to the trained Keras checkpoint.

SignMate Project Example: Verified in Chapter 9 evaluation audit.

Q102 **[Cat R] What was the most challenging technical bug you encountered?**

Answer: The double-normalization bug where dividing pixel values by 255 in external preprocessing destroyed inference accuracy because MobileNet already had an internal rescaling layer.

Deeper Technical Explanation: Debugging involved inspecting the raw tensor values entering the first convolution layer and realizing they were near zero.

SignMate Project Example: Fixed by passing unscaled float32 pixel values (0–255) to the model.

Q103 **[Cat S] What are the primary limitations of the current system?**

Answer: 1) Classifies static alphabet signs only (no continuous dynamic grammar); 2) Maximum 4 participants due to full mesh bandwidth; 3) Optimized primarily for right-handed gestures; 4) Requires good room lighting.

Deeper Technical Explanation: Each limitation is technically documented with its engineering cause in Chapter 19.

SignMate Project Example: Continuous translation requires temporal sequence models like Transformers.

Q104 **[Cat S] Why is participant capacity limited to 4 in a full mesh?**

Answer: In a full mesh, each client must upload N-1 video streams. For 4 participants, each user uploads 3 streams (~3.6 Mbps). Beyond 4 participants, client CPU and upstream bandwidth saturate quickly.

Deeper Technical Explanation: Scaling to 10+ participants requires an SFU (Selective Forwarding Unit) media server.

SignMate Project Example: Formula: N * (N - 1) / 2 connections.

Q105 **[Cat S] How would you implement continuous sign language in the future?**

Answer: By replacing single-frame classification with a spatio-temporal model (such as a 3D CNN, ConvLSTM, or Video Vision Transformer) that analyzes 30-frame video clips over time.

Deeper Technical Explanation: This would allow the model to recognize motion trajectory, two-handed interaction, and facial expressions.

SignMate Project Example: Outlined in Chapter 19 Future Roadmap.

Q106 **[Cat S] How would you scale SignMate to support 20 or 50 participants per room?**

Answer: By upgrading from a full-mesh topology to an SFU (Selective Forwarding Unit) such as MediaSoup or LiveKit.

Deeper Technical Explanation: In an SFU architecture, each client uploads only 1 video stream to the cloud server, which forwards it to other participants, reducing client upload load from O(N) to O(1).

SignMate Project Example: Documented in Chapter 19.

Q107 **[Cat T] Why did you switch away from the MediaPipe landmarks + GRU model?**

Answer: The GRU model suffered from cascading failures: whenever MediaPipe lost a single finger landmark due to lighting or occlusion, the entire sequence classification failed.

Deeper Technical Explanation: MobileNetV3Small processes raw image pixels directly, making it significantly more resilient to finger overlap and lighting variations.

SignMate Project Example: Documented in Chapter 9.

Q108 **[Cat T] Why did you change from Flutter to a React web application?**

Answer: To eliminate installation friction. A Deaf user or hearing doctor should not have to download an APK and configure permissions to join a video call.

Deeper Technical Explanation: A web application opens instantly in any browser via a URL link, offering superior accessibility.

SignMate Project Example: Documented in Chapters 2 and 18.

Q109 **[Cat T] Why didn't you use ZEGOCLOUD as originally planned?**

Answer: ZEGOCLOUD is a paid commercial proprietary SDK that charges per-minute fees and creates vendor lock-in.

Deeper Technical Explanation: Building a native WebRTC mesh gave us complete ownership, zero recurring costs, and deep architectural understanding of real-time protocols.

SignMate Project Example: Documented in Chapter 7 and Chapter 18.

Q110 **[Cat T] Is the Google Gemini integration fully operational in your live deployment?**

Answer: No. Gemini is architected as an interface in src/services/GeminiService.ts for Phase 5 cloud proxy deployment. The live site currently uses our fast client-side 95,000-word dictionary service.

Deeper Technical Explanation: This was an intentional architectural decision to avoid exposing Gemini API keys in the client browser.

SignMate Project Example: Verified in src/gemini/geminiSuggestionsService.ts.

⚠️ **Examiner Trap / Status Alert:** Never pretend Gemini is live in production. Always state it is an architected Phase 5 enhancement.

Chapter 21: Rapid Revision Sheet & Pre-Viva Cheat Sheet

21.1 Master Project Quick-Reference Table

TOPIC / METRIC	VERIFIED PROJECT VALUE (MEMORIZE THESE EXACT FACTS)
Project Name & Tagline	**SignMate** — AI-Assisted Inclusive Communication
Candidate & Register No.	**Muhammed Rashid KP** (Register No: **TVE25MCA-2046**)
Institution & Program	College of Engineering Trivandrum (TVE) — Master of Computer Applications (MCA)
Frontend Architecture	React 19 + TypeScript + Vite 8 + Tailwind CSS 3 (Hosted on Vercel CDN)
Live Deployment URL	`https://signmate-ashy.vercel.app/`
Video Communication	Browser-native **WebRTC Full Mesh** (Maximum **4 participants**, 6 P2P connections)
Signaling Mechanism	3-Tier Hybrid: Firebase Realtime Database (Primary) + FastAPI (LAN) + BroadcastChannel (Local Tabs)
AI Model Architecture	**MobileNetV3Small** Transfer Learning (Pretrained ImageNet Base + Dense 128 Head + Softmax 27)
Model Checkpoint & Format	`SignMate_stage1_best.keras` (5.27 MB) & `SignMate_MobileNetV3Small.tflite` (3.88 MB)
Input Tensor Shape	**[1, 160, 160, 1]** (Batch: 1, Height: 160, Width: 160, Grayscale Channels: 1, float32, Range: 0–255)
Output Tensor Shape	**[1, 27]** (Softmax probability distribution across 27 classes)
Class Index 0	**Background / Neutral / No-Hand** (Maps to empty string `""` to prevent false letters)
Class Indices 1 to 26	Letters **A through Z** (Standard ISL static manual fingerspelling)
Total Dataset Size	**17,113 grayscale images** (12,845 train [~75%] + 4,268 test [~25%])
Model Parameters	**1,171,139 total parameters** (77,339 trainable in Stage 1; 939,120 frozen ImageNet base)
Validation Best Metric	**100.00% validation accuracy** at Epoch 4 (val_loss: 0.0078; early stopping at Epoch 7)
Independent Test Metrics	**Accuracy: 97.00%** | **Loss: 0.1045** | **Macro F1: 0.9704** | **Macro Recall: 0.9703**
Language Assistance	Client-side **95,000-word dictionary engine** (`dictionaryService.ts`) with <2ms prefix search
Speech Synthesis	Browser-native **Web Speech API** (`window.speechSynthesis`)

21.2 Ten Difficult Questions with Punchy Answers

1. **Why MobileNetV3Small?** "Because it has only 1.17M parameters, runs inference in ~34ms on standard CPUs, and fits within 3.88 MB for edge deployment."
2. **Why is Class 0 essential?** "Without Class 0 (background), the model is forced to predict a letter whenever the camera views an empty room or resting hands."
3. **Why not divide pixel inputs by 255?** "MobileNetV3Small was trained with `include_preprocessing=True`, which rescales internally. Dividing by 255 outside causes catastrophic double-normalization."
4. **Why 4 participants max?** "Full mesh requires $N(N-1)/2$ connections. For 4 users, each maintains 3 connections (6 total in room), reaching the bandwidth limit for client uploads without an SFU."
5. **How did you solve signaling glare?** "By comparing client IDs lexicographically (`clientId_A < clientId_B`) so only one peer initiates the offer."
6. **What does Vercel host?** "Vercel hosts the static React bundle and assets. It does not run Python. The Python AI server runs locally or on a VPS."
7. **Does Gemini recognize signs?** "No. Gemini is a text LLM. MobileNetV3Small performs sign classification. Gemini is an architected Phase 5 text completion interface."
8. **Why did you abandon GRUs?** "MediaPipe landmark coordinates suffered from cascading failures when finger joints were occluded. CNNs process raw image textures directly."
9. **Why 100% val accuracy vs 97% test accuracy?** "Validation data shared identical camera environments with training batches; the independent test set featured completely separate physical signers."
10. **Is this full ISL translation?** "No. This is static alphabet fingerspelling recognition with dictionary word assistance. Full continuous ISL requires temporal dynamic modeling."

21.3 "Read This Before Entering the Viva" (1-Page Emergency Cheat Sheet)

THE 6 GOLDEN RULES TO DEFEND SIGNMATE

1. **Never say "continuous sign language translation":** Always say *"isolated static alphabet recognition (fingerspelling) with word prediction assistance"*.
2. **Know your model by heart:** *MobileNetV3Small*, transfer learning on ImageNet, input *[1, 160, 160, 1]* grayscale float32 (range 0–255), 27 classes, *1.17M total parameters*.
3. **Know your test metrics:** *97.00% test accuracy*, *0.1045 test loss*, *0.9704 macro F1* on 4,268 test images.
4. **Know your WebRTC numbers:** *Up to 4 participants* in a *full mesh*, giving *6 connections* in the room ($N(N-1)/2$). STUN via Google; signaling via Firebase RTDB.
5. **Be honest about Gemini:** Gemini is *designed as a Phase 5 proxy interface*; production currently uses our fast client-side *95,000-word dictionary service*.
6. **Be confident about stack changes:** Migrating from Flutter and ZEGOCLOUD to React and WebRTC was a deliberate engineering improvement to achieve *zero-installation web accessibility* and *eliminate paid vendor lock-in*.

21.4 Three Memorizable Presentation Scripts

1. The 30-Second Script (If the examiner says: "Be brief, tell me what you built")

"Good morning, respected examiners. I built **SignMate**, a web-based inclusive video call platform for Deaf and Hard-of-Hearing users. It allows up to four people to video call over WebRTC while an integrated deep learning model, **MobileNetV3Small**, recognizes Indian Sign Language fingerspelled alphabet signs in real time with **97% test accuracy**. The system translates manual signs into live subtitles, offers dictionary word suggestions, and vocalizes speech for hearing callers, bridging the communication barrier without requiring expensive human interpreters."

2. The 1-Minute Script (Standard Presentation Opening)

"Respected examiners, over 430 million people worldwide live with disabling hearing loss, yet mainstream video tools like Zoom or Google Meet are designed solely around spoken voice, forcing Deaf users to type in text boxes.

To solve this, I developed **SignMate**, a zero-installation web application deployed live on Vercel. SignMate establishes an encrypted, four-participant peer-to-peer WebRTC video mesh coordinated via Firebase Realtime Database. As a user signs to their webcam, our **MobileNetV3Small** computer vision model evaluates 160×160 grayscale hand crops, classifying 27 classes with **97% independent test accuracy**.

To ensure a smooth user experience, the system applies temporal smoothing and debouncing, recommends full words from a 95,000-word vocabulary, and speaks the words out loud using speech synthesis. It provides an autonomous, inclusive communication platform accessible from any modern web browser."

3. The 2-Minute Technical Script (If the examiner asks for technical depth)

"From an engineering perspective, SignMate is divided into three decoupled tiers: the presentation tier, the peer-to-peer media plane, and the machine learning inference pipeline.

The frontend is built with React 19, Vite, and TypeScript. For video conferencing, we engineered a **Full Mesh WebRTC topology** supporting up to four participants ($N=4$, yielding 6 bidirectional connections) using Google STUN servers. We solved signaling glare deterministically by comparing client IDs in Firebase Realtime Database.

Our machine learning architecture uses **MobileNetV3Small** adapted via transfer learning from ImageNet. We trained the model on 17,113 grayscale images across 27 classes, including a vital background/neutral class to eliminate false positives. The model contains 1.17 million parameters, with 77,339 trained in the classification head (Dense 128 + Dropout + Softmax 27). On an independent test set of 4,268 unseen images, the model achieved **97.00% accuracy** and a **Macro F1-score of 0.9704**.

For deployment, we converted the checkpoint into a **3.88 MB TensorFlow Lite** flatbuffer model served via an asynchronous Python FastAPI microservice at sub-50ms latency, with client-side MediaPipe landmark fallback. We also built an in-memory 95,000-word dictionary service that predicts conversational words in under 2ms. The entire frontend is live on Vercel at `signmate-ashy.vercel.app`."

Chapter 22: Comprehensive Technical Glossary

This glossary provides precise, examiner-ready definitions for over 60 core technical terms spanning WebRTC networking, machine learning, computer vision, web engineering, cloud infrastructure, and accessibility.

TECHNICAL TERM	PLAIN-ENGLISH ACADEMIC DEFINITION
API (Application Programming Interface)	A formalized digital contract enabling two different software applications to exchange data and request operations.
Backend	The server-side component of a software application responsible for data processing, database management, and heavy computational logic.
Frontend	The client-side user interface running directly inside a user's web browser, rendering visual components, graphics, and interactive buttons.
Framework	A foundational software platform that dictates architectural application flow (Inversion of Control), within which developers write custom code.
Library	A modular collection of pre-written functions, classes, and utilities that an application calls directly to solve specific sub-tasks.
WebRTC (Web Real-Time Communication)	An open-source browser API suite that enables direct, encrypted, sub-200ms peer-to-peer audio, video, and data transmission without plugins.
Signaling	The out-of-band communication process used by WebRTC peers to exchange network IP addresses, codecs, and cryptographic session descriptors before a call starts.
SDP (Session Description Protocol)	A text-based configuration payload detailing a device's media capabilities: supported video/audio codecs, network transport ports, and security keys.
SDP Offer & Answer	The two-stage negotiation handshake where Peer A offers its media capabilities, and Peer B answers with matching compatible parameters.
ICE (Interactive Connectivity Establishment)	A networking framework that tests and selects the fastest connection route between two browsers across firewalls and NAT routers.
ICE Candidate	A potential public or private IP address and port combination discovered by a browser that can be tested for peer communication.
STUN (Session Traversal Utilities for NAT)	A lightweight public server that reflects back a browser's public-facing IP address and port as seen from the open Internet.
TURN (Traversal Using Relays around NAT)	An intermediate cloud relay server that routes audio/video packets when symmetric firewalls block direct P2P connections.
Peer Connection (RTCPeerConnection)	The core WebRTC software interface that maintains an active, encrypted peer-to-peer communication link between two browsers.
Full Mesh Topology	A network architecture where every participant maintains a direct connection to every other participant, requiring N*(N-1)/2 total connections.
SFU (Selective Forwarding Unit)	A central media server architecture where each client uploads 1 stream, and the server forwards packets to other callers without re-encoding.
CNN (Convolutional Neural Network)	A deep learning neural network architecture designed specifically for 2D image analysis using sliding filter matrices to detect spatial features.
Convolution	A mathematical operation where a small filter matrix (kernel) slides over pixel arrays, multiplying and summing values to detect visual patterns like edges.
Feature Extraction	The automated process where neural network layers transform raw pixel grids into high-level geometric representations (finger joints, palm contours).
Pooling (e.g., MaxPooling, AveragePooling)	A dimensionality reduction operation that downsamples feature maps, shrinking spatial grid sizes while preserving dominant features.
GlobalAveragePooling2D	A layer that computes the mathematical mean across an entire 2D feature map, converting spatial matrices into a compact 1D feature vector.
Dense Layer (Fully Connected)	A layer where every artificial neuron connects to every neuron in the preceding layer, synthesizing high-level categorical decisions.
Dropout	A regularization technique that randomly zeroes out a fraction of neuron outputs during training to force the network to learn robust, non-redundant features.
Softmax Function	A mathematical activation function that converts unconstrained numerical scores into normalized probabilities summing to exactly 1.0 (100%).
ReLU (Rectified Linear Unit)	An activation function defined as f(x) = max(0, x), introducing non-linearity into neural networks while avoiding vanishing gradients.

TECHNICAL TERM	PLAIN-ENGLISH ACADEMIC DEFINITION
MobileNetV3Small	A state-of-the-art compact CNN developed by Google for mobile and edge vision, utilizing depthwise separable convolutions and squeeze-and-excitation blocks.
Transfer Learning	The machine learning technique of taking a model pre-trained on a massive general dataset (ImageNet) and fine-tuning it for a specific target task.
Pretrained Weights (ImageNet)	Internal numerical weight values learned by a neural network across 1.4 million natural images, serving as a rich feature initialization.
Epoch	One complete training pass where the learning algorithm evaluates every single image in the training dataset exactly once.
Batch Size	The discrete number of training samples processed simultaneously before calculating loss gradients and updating model weights.
Learning Rate	A hyperparameter controlling the magnitude of weight adjustments made by the optimization algorithm during gradient descent.
Loss Function	A mathematical equation measuring the discrepancy between a model's predicted output and the ground-truth target label.
Categorical Cross-Entropy	The standard loss function for multi-class classification, calculating the negative logarithm of the probability assigned to the true label.
Optimizer (e.g., Adam)	An algorithm that updates neural network weights based on computed gradients to minimize the loss function.
Gradient Descent	An iterative optimization algorithm that calculates partial derivatives of the loss function to find parameter values that minimize error.
Overfitting	When a model memorizes the training data too closely, achieving near-perfect training scores but failing on new, unseen test data.
Underfitting	When a model is too simple to learn the underlying patterns in the dataset, performing poorly on both training and test data.
Early Stopping	A training callback that monitors validation loss and automatically halts training when improvements plateau, preventing overfitting.
Data Augmentation	Artificially creating modified copies of training images (rotation, zoom, contrast) to expose the model to diverse visual conditions.
Validation Data	A held-out subset of data evaluated at the end of each epoch to tune hyperparameters and determine early stopping thresholds.
Test Data	An independent partition of data evaluated strictly after training concludes to objectively measure real-world generalization.
Data Leakage	An experimental error where information from the test dataset unintentionally contaminates the training pipeline, yielding falsely inflated accuracy.
Confusion Matrix	A table visualizing the performance of a classifier, mapping ground-truth classes along rows and model predictions along columns.
Accuracy	The proportion of total predictions that were correct: (True Positives + True Negatives) / Total Samples.
Precision	The proportion of positive predictions that were truly correct: TP / (TP + FP). Measures avoidance of false positive errors.
Recall (Sensitivity)	The proportion of actual target samples that were successfully detected by the model: TP / (TP + FN). Measures avoidance of false negatives.
F1-Score	The harmonic mean of precision and recall: 2 * (Precision * Recall) / (Precision + Recall), balancing both metrics.
Inference	The production execution phase where a trained, fixed neural network evaluates new input data and generates predictions.
TensorFlow Lite (TFLite)	A specialized, lightweight inference engine and FlatBuffer model format designed for low-latency machine learning execution on edge devices.
Quantization	Compressing neural network weights from 32-bit floating point down to 8-bit integers, drastically reducing file size and accelerating CPU operations.
FastAPI	A high-performance asynchronous Python web framework used to expose machine learning models as high-throughput REST APIs.

TECHNICAL TERM	PLAIN-ENGLISH ACADEMIC DEFINITION
ASGI (Asynchronous Server Gateway Interface)	The standard interface for asynchronous Python web servers (like Uvicorn), handling concurrent I/O non-blockingly.
CORS (Cross-Origin Resource Sharing)	A browser security header mechanism that controls whether a web page can make HTTP requests to a different domain name.
Firebase Authentication	Google's managed cloud identity service handling password hashing, OAuth identity tokens, and user session persistence.
Firebase Realtime Database (RTDB)	A cloud-hosted NoSQL database that synchronizes structured JSON data to connected clients via WebSockets in milliseconds.
Cloud Firestore	Google's scalable document-oriented NoSQL cloud database used for flexible, queryable application record storage.
JWT (JSON Web Token)	A compact, URL-safe, cryptographically signed token format used to transmit verified user authentication claims.
DTLS-SRTP	Datagram Transport Layer Security combined with Secure Real-time Transport Protocol, providing mandatory encryption for WebRTC media packets.
Vercel	A global cloud platform providing automated Git continuous deployment and edge Content Delivery Network (CDN) hosting for web applications.
Temporal Smoothing	A signal stabilization filter requiring an AI prediction to remain identical across consecutive video frames before committing the letter.
Debouncing	A timing mechanism that prevents sustained physical actions (like holding a gesture) from generating runaway duplicate characters.
MediaPipe Hands	Google's open-source machine learning pipeline that tracks 21 3D anatomical hand landmarks in real time from standard RGB video.
Web Speech API	A native browser API suite providing SpeechSynthesis (Text-to-Speech vocalization) and SpeechRecognition (voice transcription).